M05 — Prácticas: consolidación y despliegue elemental

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: PRÁCTICAS

AUTOR: JAIME GALLO

Índice del módulo

- P5.1 — Consolidar Blazor y la API de productos
- P5.2 — Compilar y publicar la aplicación MVC
- P5.3 — Ejecutar la publicación en Kestrel
- P5.4 — Publicar en IIS
- P5.5 — Crear y ejecutar un contenedor Docker

Cómo utilizar este manual

Tras estudiar los ejemplos y explicaciones de teoría, el alumno **construye, ejecuta y observa** las transformaciones de proyectos iniciados en módulos anteriores. El repositorio proporciona estados de referencia para comparar los resultados, no ejercicios ya realizados por el alumno.

Trabajaremos con dos aplicaciones y una API auxiliar:

Recurso	Carpeta de referencia dentro del módulo	Utilización
Aplicación B Blazor Web App Interactive Server	`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/BLAZOR/CursoWeb.Blazor`	P5.1
API auxiliar de productos	`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/API/CursoWeb.ProductosApi`	P5.1
Aplicación A MVC con EF Core/ LocalDB de M02	`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/MVC/CursoWeb.Mvc`	P5.2–P5.5

P5.1 utiliza Blazor B y su API; P5.2–P5.5 reutilizan MVC A. El puesto requiere Windows, Visual Studio/.NET 8 y SQL Server Express LocalDB para el CRUD. IIS precisa Windows y Hosting Bundle 8; Docker, un motor de contenedores Linux. **No se declara comprobado lo que carece de infraestructura.**

Tiempo de la jornada

Punto	Teoría	Práctica
5.1 Consolidación Blazor	10 min	90 min
5.2 Ejecutar/publicar	15 min	35 min
5.3 Kestrel	10 min	45 min
5.4 IIS	15 min	55 min
5.5 Docker	10 min	45 min
Subtotal	**60 min**	**270 min**

La jornada reserva otros **30 minutos** para dudas. Las ampliaciones y el diagnóstico extendido son autónomos. Usa una terminal por proceso, verifica las rutas de ejecución y conserva el estado final de cada ejercicio antes de avanzar.

Itinerario presencial de 270 minutos: P5.1.1–P5.1.7 (90 min); P5.2.1–P5.2.6 (35 min); P5.3.1–P5.3.5 (45 min); P5.4.1–P5.4.8 (55 min); P5.5.1–P5.5.5 y P5.5.10–P5.5.13 (45 min, reconstrucción guiada del Dockerfile incluida). Los demás pasos numerados, las variantes de diagnóstico y las comparaciones adicionales son ampliaciones autónomas, no obligaciones encubiertas para la misma jornada. La estimación presupone Visual Studio, SQL Server Express LocalDB, IIS con Hosting Bundle y Docker Desktop **preinstalados y operativos**; si la infraestructura no está preparada, el docente debe adaptar la sesión, no declarar una ejecución ficticia.

P5.1. Consolidar el catálogo Blazor y la API HTTP

Objetivo, estado inicial y resultado previsto

Partimos de la aplicación Blazor Interactive Server que funciona al finalizar M04 y de la API de productos construida allí. Queremos comprobar toda la cadena: pedir productos, mostrar datos, intentar crear un producto inválido, crear uno válido y consultar de nuevo el servidor. No añadiremos Identity/JWT, no crearemos otra aplicación Blazor y no conectaremos esta API de memoria a LocalDB.

El estado inicial contiene:

- `CursoWeb.Blazor/Program.cs`, que registra `IProductosApiService` como cliente HTTP tipado.
- `Components/Pages/CatalogoRemoto.razor`, que consulta y permite crear productos.
- `Services/IProductosApiService.cs` y `Services/ProductosApiService.cs`, que encapsulan GET/POST.
- `CursoWeb.ProductosApi/Program.cs`, con `GET /api/productos` y `POST /api/productos`.

El resultado observable de salida será una pantalla que obtiene dos productos iniciales, incorpora un tercero sin alterar la API, informa de errores y permite realizar una consulta explícita cuando el catálogo del servidor haya cambiado.

Paso P5.1.1. Recuperar las aplicaciones correctas

Desde la raíz del repositorio del curso abre dos terminales de PowerShell. En la primera, identifica y compila la API de productos:

```
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
```

En la segunda compila el proyecto Blazor:

```
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj
```

Qué estamos comprobando: ambos proyectos tienen sus dependencias y sintaxis válidas. Todavía no sabemos si hay comunicación HTTP porque no hemos iniciado el servidor. Si falla la compilación, comprobar que el SDK 8 aparece con `dotnet --list-sdks`, que los comandos parten de la raíz y que la restauración NuGet ha funcionado.

Paso P5.1.2. Iniciar la API y observar su contrato

En la primera terminal:

```
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

Sin cerrar la terminal, desde un navegador visita `http://localhost:5081/api/productos`. Debes ver un arreglo JSON con al menos Cuaderno y Lápiz en una API recién iniciada. Comprueba además un producto que no existe:

```
curl.exe -i http://localhost:5081/api/productos/999
```

Debe devolver **404 Not Found**; no significa que la API esté apagada. El servidor ha recibido la solicitud y ha indicado que ese identificador no corresponde a un producto existente. Si la URL de lista no responde, comprobar que el proceso sigue activo y que no hay otra API utilizando el puerto.

Paso P5.1.3. Arrancar Blazor y comprobar la primera consulta

En la segunda terminal:

```
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj --no-
launch-profile --urls http://localhost:5092
```

Abre `http://localhost:5092/catalogo-remoto`. Debe mostrarse el catálogo remoto; la página local `/productos` conserva el trabajo de M03 y no sustituye al endpoint HTTP de M04.

Abre `appsettings.json` del proyecto Blazor y comprueba que la dirección de productos es:

```
{
  "ProductosApi": {
    "BaseUrl": "http://localhost:5081/"
  },
  "IdentityJwtApi": {
    "BaseUrl": "http://localhost:5086/"
  }
}
```

No hace falta ejecutar la API Identity/JWT: su dirección existe en configuración porque se conservó la ampliación, pero `/catalogo-remoto` consume `ProductosApi`. Si el catálogo remoto no se carga, llama primero directamente a la API y revisa después `ProductosApi:BaseUrl`.

Paso P5.1.4. Añadir la consulta explícita «Consultar servidor»

La versión de trabajo de M04 obtenía productos durante `OnInitializedAsync`. El archivo de partida se conserva como **variante inicial fuera del proyecto compilable**, en `PROYECTO/VARIANTES/P5.1_ANTES/Components/Pages/CatalogoRemoto.razor`. Como el proyecto acumulativo M05 ya contiene el resultado de referencia, **no añadas por segunda vez** el botón ni el método. Para construir el cambio desde el estado inicial de M04, recupera de forma reversible la variante anterior desde la raíz del repositorio, usando PowerShell:

```
$pagina = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\Components\Pages\CatalogoRemoto.r
azor"
$partida = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.1_ANTES\Components\Pages\CatalogoRemoto.raz
or"
$respaldo = "$pagina.p514-referencia"
if (Test-Path $respaldo) { throw "Ya existe un respaldo P5.1.4; revísalo antes de continuar." }
```

```
Copy-Item $pagina $respaldo
Copy-Item $partida $pagina -Force
```

Ahora el archivo que compila es el estado **anterior al botón**. Queremos permitir una segunda consulta sin reconstruir toda la página. En `Components/Pages/CatalogoRemoto.razor`, debajo del párrafo introductorio, añade:

```
<button id="recargar-api"
        disabled="@cargando"
        @onclick="ConsultarAsync">
    Consultar servidor
</button>
```

La propiedad `disabled` evita iniciar consultas simultáneas desde el mismo botón mientras una operación ya está cargando. No evita por sí sola cualquier concurrencia posible; es una simplificación adecuada para este laboratorio.

Después localiza el método `OnInitializedAsync`. Sustitúyelo por la siguiente delegación y método de consulta. **No crees otro `OnInitializedAsync`**: debe existir una sola implementación del mismo ciclo de vida.

```
protected override async Task OnInitializedAsync() =>
    await ConsultarAsync();

private async Task ConsultarAsync()
{
    cargando = true;
    error = null;
    try
    {
        productos = (await CatalogoApi.ObtenerAsync()).ToList();
    }
    catch (HttpRequestException)
    {
        error = "No se pudo obtener el catálogo remoto.";
    }
    catch (TaskCanceledException)
    {
        error = "La petición superó el tiempo disponible.";
    }
    catch (System.Text.Json.JsonException)
    {
        error = "La API no devolvió un catálogo JSON válido.";
    }
    finally
    {
        cargando = false;
    }
}
```

El ejemplo supone que el componente conserva sus campos de M04:

```
private List<ProductoDto> productos = [];
private bool cargando = true;
private string? error;
```

El código final de referencia contiene esos campos y el método ya integrado. Compila y abre la página. Pulsa «Consultar servidor»: debe seguir mostrando la lista. La intención no es producir un

nuevo diseño de UI, sino comprobar que el mismo servicio puede reutilizarse desde el ciclo de vida y desde un evento del usuario.

Paso P5.1.5. Crear y validar sin introducir duplicidades

La página de M04 conserva el formulario Nombre/Precio y su método `CrearAsync`. Antes de pulsar «Crear en API», deja el nombre vacío e indica precio 0: el componente debe responder **«Indica un nombre y un precio mayor que cero»** sin enviar una creación válida.

Después introduce:

Campo	Valor
Nombre	Regla
Precio	1.75

Pulsa «Crear en API». El servicio tipado debe enviar POST y recibir un `ProductoDto` con identificador asignado por el servidor. Debe aparecer el mensaje de creación y el producto debe añadirse a la lista. Desde otra terminal solicita directamente la lista de la API y comprueba que también incluye Regla.

Distinción importante: si el mensaje aparece porque el DTO devuelto se incorporó a la lista local, todavía no hemos probado que un segundo cliente consulte el estado del servidor. Esa es la razón del siguiente paso.

Paso P5.1.6. Comparar estado local y servidor

Manteniendo abierta Blazor, añade otro producto a la API directamente. Crea en una carpeta temporal un fichero `producto-prueba.json`:

```
{
  "nombre": "Archivador",
  "precio": 5.25
}
```

Ejecuta desde la carpeta que contiene ese archivo:

```
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" --
data-binary "@producto-prueba.json"
```

La API responderá 201. **Antes** de pulsar «Consultar servidor», la pantalla Blazor no tiene por qué mostrar Archivador: no ha recibido ninguna notificación push. Al pulsar el botón de consulta, la lista debe incluirlo. Es una demostración observable de la diferencia entre estado mostrado y fuente de datos.

Detén **solo la API**, sin cerrar la ventana Blazor. Pulsa «Consultar servidor» de nuevo: aparecerá un mensaje de fallo de red o de tiempo, según la causa observada. Vuelve a iniciar la API; la consulta recupera los productos iniciales, porque esta API utiliza memoria. No interpretes la desaparición de Regla/Archivador como una pérdida de SQL Server ni como un error de Blazor.

Paso P5.1.7. Completar la cadena de diagnóstico

Recorre esta matriz antes de considerar la integración terminada:

Comprobación	Acción	Resultado esperado
API disponible	GET directo a `/api/productos`	200 JSON
Identificador inexistente	GET a `/api/productos/999`	404
Blazor disponible	Abrir `/catalogo-remoto`	HTML y componente interactivo
Alta inválida desde UI	Nombre vacío, precio 0	Rechazo local
Alta válida desde UI	Regla, 1.75	Producto creado
Alta desde cliente externo	POST de Archivador	201 y nuevo DTO
Estado no actualizado automáticamente	Observar Blazor antes de consultar	No asumir push
Actualización explícita	Pulsar «Consultar servidor»	Catálogo actualizado
API detenida	Consultar de nuevo	Mensaje de error
API reiniciada	Volver a consultar	Datos de inicio

Si los resultados no coinciden, diagnostica empezando en la API directa, después servicio tipado y finalmente el componente. No copies un `Program.cs` ajeno para resolver un problema de puerto. Esta matriz describe escenarios observables que el alumno puede ejecutar; la confirmación técnica automática se documenta fuera del material docente.

Restauración del proyecto para seguir con los siguientes puntos: si construiste el componente desde la variante de P5.1.4, vuelve al estado acumulativo de referencia que conserva la implementación de errores HTTP y JSON más completa. Detén Blazor y, en la misma terminal PowerShell donde definiste las variables, ejecuta:

```
if (-not (Test-Path $respaldo)) { throw "No existe el respaldo original de P5.1.4." }
Copy-Item $respaldo $pagina -Force
Remove-Item $respaldo
```

Reinicia Blazor y comprueba el botón «Consultar servidor». Los ejercicios P5.1.8 y siguientes trabajan sobre este estado acumulativo; el cambio de P5.1.4 no se duplica.

Paso P5.1.8. Diferenciar ausencia de resultados y error de comunicación

Relación con la teoría: apartados 5.1.10, 5.1.12 y 5.1.13. Hemos aprendido que recibir un arreglo vacío no es lo mismo que no recibir respuesta HTTP. La interfaz inicial representaba la lista mediante un `<ul>`; si el servidor devolvía `[]`, el usuario veía una lista sin elementos, pero no una explicación del estado.

Situación inicial. El componente `M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/BLAZOR/CursoWeb.Blazor/Components/Pages/CatalogoRemoto.razor` ya posee la propiedad `productos`, el indicador `cargando` y la propiedad `error`. No añadiremos otra llamada al servidor ni otro servicio. Cambiaremos únicamente la presentación de una colección recibida satisfactoriamente.

Archivo que debes editar: `PROYECTO/BLAZOR/CursoWeb.Blazor/Components/Pages/CatalogoRemoto.razor`, siempre interpretando esa ruta desde la carpeta `M05_CONSOLIDACION_DESPLIEGUE`.

Busca el bloque de renderizado. Después de la rama que comprueba `error`, y antes del `else` que dibuja el `<ul>`, introduce esta rama:

```
else if (productos.Count == 0)
{
    <p id="estado-api">
        El servidor respondió correctamente, pero no hay productos disponibles.
    </p>
}
```

El orden es parte del ejercicio, no un detalle cosmético. Si pusiéramos la comprobación de lista vacía antes del error, podríamos mostrar «no hay productos» cuando la conexión ha fallado y la lista simplemente no llegó a cargarse. Si la situáramos después de la rama `else` final, sería código inalcanzable dentro de esa secuencia de condiciones.

El conjunto de ramas, una vez construido, debe conservar este orden lógico:

```
@if (cargando)
{
    <p id="estado-api" role="status">Cargando catálogo remoto...</p>
}
else if (error is not null)
{
    <p id="error-api" role="alert">@error</p>
}
else if (productos.Count == 0)
{
    <p id="estado-api">
        El servidor respondió correctamente, pero no hay productos disponibles.
    </p>
}
else
{
    <p id="estado-api">Catálogo remoto disponible</p>
    <ul id="productos-api">
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
        }
    </ul>
}
```

Este bloque usa los campos que ya creó M04; **no** debes duplicar sus declaraciones ni insertar otra propiedad con el mismo nombre. En el árbol de proyecto de referencia la transformación ya aparece aplicada. Para reconstruirla desde el estado anterior, utiliza `PROYECTO/VARIANTES/P5.1_ANTES/Components/Pages/CatalogoRemoto.razor` y vuelve a incorporar primero los cambios de P5.1.4 y después los de este apartado.

Compila el proyecto:

```
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj
```

Resultado intermedio esperado: el proyecto compila y, con la API normal, sigue mostrando Cuaderno y Lápiz. Esto confirma que la rama nueva no ha roto el escenario conocido. Todavía no confirma que el mensaje de colección vacía se muestre realmente. Ese escenario se ejecuta a continuación.

Razonamiento solicitado al alumno: explica por qué una rama de error tiene prioridad sobre una rama de lista vacía y por qué ninguna de las dos demuestra, por sí sola, que el proceso de Blazor está detenido. La respuesta debe identificar la diferencia entre una respuesta HTTP válida, un error de transporte y la decisión de renderizado.

Paso P5.1.9. Preparar una variante reversible del mismo servidor

Relación con la teoría: 5.1.3, 5.1.12 y 5.1.16. Para estudiar un catálogo vacío y un cuerpo JSON inválido no hace falta crear una tercera aplicación ni modificar el proyecto Blazor. Podemos sustituir temporalmente **un solo archivo** de la API por una variante completa, incluida en el repositorio, con modos de salida seleccionados mediante configuración.

El archivo de referencia para esta ampliación se encuentra en:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/VARIANTES/P5.1_DIAGNOSTICO/Program.cs`

No es otro proyecto ni un resultado que el estudiante tenga que inventar. Contiene los mismos endpoints de la API de M04; únicamente cambia la respuesta de `GET /api/productos` cuando se elige expresamente un escenario de diagnóstico. Por defecto, sigue atendiendo el GET normal.

Preparación segura. Detén el proceso de la API con Ctrl+C. Puedes mantener Blazor abierto. Abre una terminal PowerShell en la raíz del repositorio y ejecuta las instrucciones completas siguientes:

```
$origen = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\Program.cs"
$variante = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.1_DIAGNOSTICO\Program.cs"
$respaldo = "$origen.p51-copia"

Copy-Item $origen $respaldo -Force
Copy-Item $variante $origen -Force

dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
```

No suprimas la copia de seguridad hasta haber terminado los escenarios. Si al copiar el archivo aparece un error de acceso, comprueba que el proceso de API se ha detenido. Si falla la compilación, restaura inmediatamente el original con el procedimiento final de este apartado y comprueba que sigues partiendo de la misma versión del curso.

Qué contiene la variante. Localiza `ObtenerCatalogo` y observa la selección de una respuesta en función del valor `Laboratorio:EscenarioCatalogo`:

```
IResult ObtenerCatalogo()
{
    if (escenarioCatalogo == "vacio")
        return Results.Ok(Array.Empty<ProductoApi>());

    if (escenarioCatalogo == "json-invalido")
        return Results.Text("{\"productos\":[", "application/json");

    return Results.Ok(productos.Values.OrderBy(p => p.Id));
}
```

```
app.MapGet("/api/productos", ObtenerCatalogo);
```

El bloque depende del resto del `Program.cs` **completo** que se ha entregado: la variable de configuración, el diccionario de productos y el tipo `ProductoApi` ya aparecen en él. No copies solo estas líneas en otro proyecto vacío. `Results.Ok(Array.Empty<ProductoApi>())` constituye una respuesta 200 con colección válida vacía. `Results.Text` entrega, intencionadamente, contenido malformado con tipo de medio JSON para mostrar que un HTTP 200 tampoco garantiza poder deserializar el cuerpo. Esta variante se reserva al entorno de aprendizaje.

Paso P5.1.10. Ejecutar tres respuestas sin modificar el componente Blazor

Escenario A — servidor y catálogo normales. En la terminal de API, desde la raíz del repositorio:

```
$env:Laboratorio__EscenarioCatalogo = "normal"
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

Desde otra terminal ejecuta:

```
curl.exe -i http://localhost:5081/api/productos
```

Debe observarse HTTP 200 con un arreglo que contiene Cuaderno y Lápiz. Pulsa «Consultar servidor» en `http://localhost:5092/catalogo-remoto` y comprueba que aparecen los productos. El resultado confirma que copiar el archivo de variante no cambia el comportamiento normal mientras la configuración sea `normal`.

Escenario B — servidor disponible, lista vacía. Detén la API con Ctrl+C y, en su terminal, ejecuta:

```
$env:Laboratorio__EscenarioCatalogo = "vacio"
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

Comprueba otra vez la URL directa. El código de respuesta seguirá siendo **200**; el cuerpo debe representar `[]`. En Blazor pulsa «Consultar servidor». Ahora debe aparecer el mensaje «El servidor respondió correctamente, pero no hay productos disponibles». No debe aparecer el mensaje de error de comunicación. Comprueba que el botón sigue habilitándose después de la consulta.

Razón: el servidor funciona y está cumpliendo un contrato de colección, pero esa colección no tiene elementos. Si la interfaz lo presentara como fallo, enseñaría una conclusión falsa.

Escenario C — servidor disponible, contenido JSON incorrecto. Vuelve a detener la API y ejecuta:

```
$env:Laboratorio__EscenarioCatalogo = "json-invalido"
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

Consulta mediante `curl.exe -i` el mismo endpoint. Observa que puede responder **HTTP 200**, pero el cuerpo no representa una lista JSON deserializable. En Blazor pulsa «Consultar servidor» y comprueba que se muestra el aviso «La API no devolvió un catálogo JSON válido». La rama de error tiene prioridad sobre la condición de lista vacía.

Razón: el transporte y el código de respuesta HTTP han funcionado; el contrato de contenido no. Cambiar de puerto al azar no corregiría el problema porque la respuesta llega al destino correcto.

Restauración obligatoria del ejercicio. Detén la API y ejecuta:

```
$origen = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\Program.cs"
$respaldo = "$origen.p51-copia"

Copy-Item $respaldo $origen -Force
Remove-Item $respaldo
Remove-Item Env:\Laboratorio__EscenarioCatalogo -ErrorAction SilentlyContinue

dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
```

Arranca otra vez la API en su modo normal y pulsa «Consultar servidor». Deben reaparecer Cuaderno y Lápiz. Si decides conservar el proyecto para seguir practicando, la restauración garantiza que el siguiente alumno no encuentre una API que devuelve JSON defectuoso por una prueba anterior.

Paso P5.1.11. Comprobar un rechazo de servidor que el formulario local no puede ocultar

Relación con la teoría: 5.1.5 y 5.1.14. El componente Blazor valida un nombre y un precio antes de enviar el POST. Pero otro cliente HTTP puede saltarse la interfaz; el servidor debe validar por sí mismo.

Con la API normal arrancada, crea un archivo temporal `producto-invalido.json`, fuera de la carpeta de código, con este contenido íntegro:

```
{
  "nombre": "     ",
  "precio": 0
}
```

Desde la carpeta donde guardaste el JSON, ejecuta:

```
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" --
data-binary "@producto-invalido.json"
```

La API debe devolver **400 Bad Request**. Vuelve a solicitar `GET /api/productos` y comprueba que no aparece un producto nuevo creado por esa petición. Después vuelve a Blazor, intenta enviar el mismo nombre y precio mediante el formulario y observa el rechazo local.

El ejercicio demuestra dos validaciones con responsabilidades diferentes: la interfaz impide un envío evitable, pero el endpoint rechaza de manera independiente cualquier cliente que incumpla el contrato. No debemos considerar la API protegida solo porque la pantalla esté programada correctamente.

Cierre razonado del caso: enumera qué pruebas demostrarían que la API está detenida, que ha respondido con una colección válida vacía, que ha respondido 200 con JSON malformado y que ha rechazado un POST. En los cuatro casos el aspecto inicial puede parecerse, pero el diagnóstico correcto exige comparar estado HTTP, cuerpo y mensaje del componente.

Paso P5.1.12. Mejorar la información ante una ruta HTTP inexistente

Relación con la teoría: 5.1.25 y 5.1.26. El proyecto inicial informaba de un fallo HTTP mediante un mensaje general. Ahora queremos distinguir una conexión que no ha recibido respuesta de un servidor que sí ha contestado con 404.

Abre `PROYECTO/BLAZOR/CursoWeb.Blazor/Components/Pages/CatalogoRemoto.razor`. Dentro de `ConsultarAsync`, localiza la primera captura de `HttpRequestException` y sitúa **inmediatamente antes** una captura filtrada por código de estado:

```
catch (HttpRequestException ex)
    when (ex.StatusCode == System.Net.HttpStatusCode.NotFound)
{
    error = "La API respondió 404: no existe la ruta del catálogo.";
}
catch (HttpRequestException)
{
    error = "No se pudo obtener el catálogo remoto.";
}
```

No introduzcas estas capturas en `CrearAsync`: estamos modificando exclusivamente la clasificación de errores del GET. El resto del componente sigue controlando `cargando` en `finally`, y conserva los casos de cancelación y error JSON. No agregues otro `ConsultarAsync` ni repitas las llamadas al servicio.

Por qué importa el orden: si el bloque general apareciera primero, absorbería también el 404. El filtro `when` selecciona únicamente la excepción cuyo código conocido es `NotFound`. Si no hay respuesta HTTP útil, el filtro no presupone un 404 y puede ejecutarse la captura general.

Compila la aplicación Blazor desde la raíz:

```
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj
```

Arranca la API en modo normal y Blazor como en P5.1.2–P5.1.3. Con Cuaderno y Lápiz disponibles, la nueva captura no debería alterar el resultado. El código final de referencia de M05 incluye ya esta modificación; el archivo inicial del módulo M04 permite reconstruirla desde el estado anterior.

Paso P5.1.13. Comparar HTTP 404 y HTTP 500 desde el mismo servicio

Estado inicial: Blazor contiene la captura especializada y la API original está restaurada. Detén únicamente la API; no hace falta detener el componente Blazor. Volveremos a usar el archivo completo `PROYECTO/VARIANTES/P5.1_DIAGNOSTICO/Program.cs`, que permite seleccionar cinco escenarios sin construir una API diferente.

Desde una terminal PowerShell situada en la raíz del repositorio:

```
$origen = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\Program.cs"
$variante = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.1_DIAGNOSTICO\Program.cs"
$respaldo = "$origen.p513-copia"
```

```
Copy-Item $origen $respaldo -Force
Copy-Item $variante $origen -Force
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
```

Escenario 404: en la terminal de la API ejecuta:

```
$env:Laboratorio__EscenarioCatalogo = "no-encontrado"
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

Desde otra terminal solicita `curl.exe -i http://localhost:5081/api/productos`. La respuesta prevista es 404. En Blazor pulsa «Consultar servidor» y comprueba el mensaje «La API respondió 404: no existe la ruta del catálogo». A diferencia de una conexión rechazada, aquí existe una respuesta HTTP con estado conocido.

Escenario 500: detén la API con Ctrl+C, cambia la variable y vuelve a iniciar el mismo proyecto:

```
$env:Laboratorio__EscenarioCatalogo = "error-servidor"
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

La solicitud directa debe devolver HTTP 500. El componente mostrará su aviso general de fallo al obtener el catálogo. No debe informar falsamente de que la ruta no existe, porque el servidor ha contestado con un código diferente.

Escenario sin respuesta: detén la API sin cerrar Blazor y vuelve a pulsar el botón. Ahora no hay un servidor que genere 404 ni 500 en ese puerto. El componente debe mostrar el aviso general; la prueba directa también fallará por no disponer de una respuesta HTTP de la API.

Restauración final obligatoria: después de detener la API ejecuta:

```
$origen = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\Program.cs"
$respaldo = "$origen.p513-copia"

Copy-Item $respaldo $origen -Force
Remove-Item $respaldo
Remove-Item Env:\Laboratorio__EscenarioCatalogo -ErrorAction SilentlyContinue

dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
```

El resultado final debe ser de nuevo el catálogo normal, con Blazor capaz de consultar Cuaderno y Lápiz. La modificación permanente del componente consiste únicamente en ofrecer un mensaje más preciso cuando recibe un 404 real. Los modos de fallo pertenecen a la variante reversible, no al comportamiento predeterminado de la API.

Paso P5.1.14. Reconstruir el recorrido completo de consulta desde la interfaz

Origen conceptual: 5.1.27–5.1.29. **Objetivo:** reconocer qué partes de una operación pertenecen al navegador, al componente Interactive Server, al servicio tipado y a la API sin crear una

aplicación adicional. La secuencia se basa en el código real de B y de la API auxiliar entregados en M04.

Prepara Visual Studio con dos proyectos abiertos. En la aplicación Blazor localiza `Components/Pages/CatalogoRemoto.razor` y `Services/ProductosApiService.cs`. En la API localiza `Program.cs`. No edites todavía ninguno: primero identifica sus puntos de conexión.

En `CatalogoRemoto.razor`, busca la directiva que resuelve el servicio:

```
@inject IProductosApiService CatalogoApi
```

Esta línea no envía una petición por sí sola. Solicita que el sistema de inyección de dependencias proporcione un objeto capaz de cumplir el contrato. Continúa hasta `OnInitializedAsync`, que delega la consulta a `ConsultarAsync`. Localiza exactamente la llamada `CatalogoApi.ObtenerAsync()`: aquí el componente solicita datos, pero todavía no conoce cómo se forma el mensaje HTTP.

Abre ahora el servicio concreto. Comprueba que utiliza `GetFromJsonAsync<List<ProductoDto>>("api/productos")`. Esta operación utiliza una dirección base configurada durante el arranque de Blazor. Dirígete a `Program.cs` de B y busca el registro de cliente HTTP tipado, así como la lectura de `ProductosApi:BaseUrl`.

Explicación de la cadena: el componente solicita una operación de dominio; el servicio utiliza HttpClient; la API atiende GET; el servicio deserializa JSON; el componente asigna su colección y Razor genera la representación. Que aparezcan muchas clases no exige crear más proyectos: cada archivo ya existe en el itinerario anterior.

Finalmente abre `Program.cs` de la API y busca el endpoint GET:

```
app.MapGet("/api/productos", () =>
    Results.Ok(productos.Values.OrderBy(p => p.Id)));
```

La colección de productos no pertenece al componente Blazor. Vive en el proceso de API y se convierte en contenido JSON de la respuesta. Anota, para cada eslabón, el archivo exacto que lo implementa.

Comprobación observable: inicia ambos servidores desde las rutas conocidas, abre Blazor y realiza un GET directo a la API. Los mismos nombres de producto deben aparecer en la página y en la respuesta JSON. Si la página falla pero el GET directo responde correctamente, no declares rota la API: continúa investigando el servicio y la configuración.

Paso P5.1.15. Encontrar la dirección equivocada sin alterar el código del servicio

Origen conceptual: 5.1.27 y 5.1.28. **Objetivo:** observar que la aplicación Blazor puede arrancar correctamente aunque su servicio HTTP apunte a un destino inadecuado.

Parte de un entorno funcional: la API atiende `http://localhost:5081/api/productos` y Blazor muestra los productos en `http://localhost:5092/catalogo-remoto`. Detén **solo Blazor**. Abre el archivo de configuración del proyecto B y guarda una copia temporal antes de editarlo:

```
$cfg = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\appsettings.json"
Copy-Item $cfg "$cfg.p515-copia" -Force
```

En ese archivo encuentra la clave `ProductosApi:BaseUrl`. Sustituye temporalmente su valor por `http://localhost:5082/`, conservando las otras propiedades y la estructura JSON. No cambies las direcciones de `IdentityJwtApi`, porque no interviene en esta demostración.

Arranca otra vez Blazor con su comando habitual:

```
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj --no-
launch-profile --urls http://localhost:5092
```

La página web puede abrirse; su consulta a la API debe fallar si no existe ningún servicio escuchando en 5082. Comprueba con el navegador o con `curl.exe` que la API real continúa funcionando en 5081. La conclusión es concreta: **la API está disponible, el componente existe, pero la configuración del cliente indica una dirección equivocada**.

No modifiques `ProductosApiService.cs` para introducir manualmente `5081` dentro de cada método. Esa corrección duplicaría información que el propio sistema de configuración está diseñado para centralizar.

Detén Blazor, restaura la copia y elimina el respaldo:

```
Copy-Item "$cfg.p515-copia" $cfg -Force
Remove-Item "$cfg.p515-copia"
```

Vuelve a iniciar Blazor. El catálogo debe cargarse normalmente. Si no lo hace, comprueba que eliminaste el cambio temporal y que el servicio de API continúa escuchando en 5081.

Paso P5.1.16. Demostrar que el componente no asigna el identificador de creación

Origen conceptual: 5.1.30–5.1.32. **Objetivo:** comparar los campos que envía el formulario con los que devuelve la API, sin introducir un identificador inventado en Blazor.

Con ambos servidores operativos, abre `Models` en el proyecto Blazor y localiza `CrearProductoDto` y `ProductoDto`. Compara sus propiedades. El primero representa el cuerpo de entrada y el segundo la representación recibida después de una operación satisfactoria.

Prepara un cuerpo de prueba fuera del proyecto:

```
{
  "nombre": "Goma",
  "precio": 0.65
}
```

Guárdalo como `nuevo-producto-p516.json` en una carpeta de trabajo. Ejecuta desde esa carpeta:

```
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" --
data-binary "@nuevo-producto-p516.json"
```

Busca en el cuerpo de respuesta el identificador asignado por el servidor. No lo presupongas: su valor depende de los productos que se hayan creado durante esa ejecución de API. Comprueba también la cabecera `Location`, cuya dirección describe el recurso creado.

A continuación consulta directamente `GET /api/productos` y localiza Goma. Desde Blazor pulsa «Consultar servidor». El producto debe aparecer en pantalla después de la nueva consulta, aunque lo creaste desde una herramienta independiente del componente.

Pregunta obligatoria del ejercicio: ¿sería correcto utilizar `productos.Count + 1` como identificador de Goma? Explica por qué la colección visual pudo estar desactualizada y por qué la respuesta 201 es la fuente de información apropiada para conocer la identidad asignada.

Paso P5.1.17. Reproducir una secuencia de estados sin confundir datos iniciales y remotos

Origen conceptual: 5.1.29 y 5.1.34. **Objetivo:** distinguir una colección creada vacía en memoria del componente, una respuesta de servidor y un estado visual actualizado.

Detén la API y reiníciala para comenzar con los dos productos iniciales del laboratorio. Abre de nuevo el catálogo remoto y observa la pantalla durante la consulta. La bandera de carga representa una operación en curso, no una lista vacía confirmada.

Cuando la API responda, la pantalla mostrará los productos iniciales. Ejecuta una creación mediante `curl.exe` desde una segunda terminal. Antes de pulsar el botón, observa que la pantalla puede conservar su representación anterior. El servicio remoto ha cambiado, pero el componente no ha recibido ninguna actualización automática.

Pulsa «Consultar servidor» y compara el resultado. Debe aparecer el nuevo recurso. A continuación detén únicamente la API y deja Blazor abierto. La pantalla puede seguir mostrando los datos recibidos antes; al solicitar una nueva consulta, se debe presentar un fallo de comunicación. No sustituyas ese error por la frase «no existen productos»: no hemos recibido una respuesta 200 con colección vacía.

Vuelve a iniciar la API, que utiliza memoria. La consulta vuelve a mostrar sus datos iniciales. Describe qué operación causó cada cambio: reinicio del servidor, petición GET, POST externo o modificación de la colección del componente. No utilices «el navegador borró la base» para explicar la desaparición de un objeto que solo vivía en la memoria del proceso de API.

Paso P5.1.18. Leer una respuesta inválida sin esconder la causa detrás del componente

Origen conceptual: 5.1.20 y 5.1.25. **Objetivo:** utilizar la variante reversible ya preparada para comprender una respuesta HTTP 200 que no satisface el contrato JSON esperado.

Si ya completaste P5.1.9, conoces cómo realizar una copia de `Program.cs` y sustituir temporalmente la API original por `PROYECTO/VARIANTES/P5.1_DIAGNOSTICO/Program.cs`. Si la variante ya estaba restaurada, repite su procedimiento de respaldo y copia. No edites las rutas normales de Blazor.

Inicia la variante de API con:

```
$env:Laboratorio__EscenarioCatalogo = "json-invalido"
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\API\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
--no-launch-profile --urls http://localhost:5081
```

Primero consulta el endpoint de la API con `curl.exe -i` y observa el cuerpo literal. Después, sin alterar el servicio tipado, pulsa «Consultar servidor» en Blazor. El error resultante se origina al

interpretar un contenido no válido para la lista de productos; no se debe atribuir a que Blazor haya intentado abrir un puerto inexistente.

Completa una comparación oral o escrita: ¿qué parte de la operación fue satisfactoria y cuál falló? La conexión llegó al servidor y este generó una respuesta HTTP 200; la interpretación JSON no pudo producir la colección esperada. Es una situación diferente de un 404, un 500 y una conexión rechazada.

Finalmente detén la API, restaura el archivo y elimina la variable de escenario siguiendo la restauración de P5.1.10. La siguiente ejecución debe comenzar de nuevo con la API normal.

Paso P5.1.19. Verificar que el error no deja bloqueado el botón de consulta

Origen conceptual: 5.1.13 y 5.1.22. **Objetivo:** observar por qué la asignación `cargando = false` pertenece al bloque `finally`.

Mantén Blazor abierto. Detén temporalmente la API y pulsa «Consultar servidor» una vez. Espera al mensaje de error. Comprueba que, cuando termina la operación fallida, el botón vuelve a estar disponible. Ahora reinicia la API y pulsa otra vez el botón. La página debe poder recuperarse y mostrar la lista.

Inspecciona el método `ConsultarAsync`. La asignación de la bandera de carga se realiza antes de la petición, y su restauración después aparece en el `finally`. Un tratamiento de excepciones que dejara la bandera activada impediría que el usuario iniciara una nueva consulta aunque el servidor ya se hubiera recuperado.

Para estudiar el fallo **sin dejar un archivo incorrecto en el proyecto**, razona sobre este fragmento deliberadamente incompleto:

```
cargando = true;
productos = (await CatalogoApi.ObtenerAsync()).ToList();
cargando = false;
```

Si `ObtenerAsync` produce una excepción, la ejecución no llega a la última asignación. El ejemplo explica por qué el proyecto real necesita gestionar los estados mediante `try`, capturas apropiadas y `finally`. No copies este fragmento defectuoso sobre el método correcto; no constituye una variante de ejecución del curso.

El resultado del ejercicio es un comportamiento observable de recuperación y una explicación de su causa, no un formulario adicional.

Paso P5.1.20. Confirmar que el servicio tipado concentra la ruta HTTP

Origen conceptual: 5.1.18–5.1.19. **Objetivo:** impedir que una corrección aparente multiplique rutas HTTP dentro de la interfaz.

En Visual Studio abre `IProductosApiService.cs`, `ProductosApiService.cs` y `CatalogoRemoto.razor`. Busca el texto `api/productos` en cada archivo. Debe encontrarse en el servicio HTTP, mientras el componente llama a las operaciones `ObtenerAsync` y `CrearAsync`.

No modifiques el proyecto para copiar la ruta dentro de `ConsultarAsync`. El reto consiste en explicar por qué ese cambio reduciría la mantenibilidad: ante un cambio de endpoint habría que buscarlo en más archivos, y cada componente podría empezar a tratar errores de manera diferente.

Comprueba también el registro de la dirección base en `Program.cs` y el valor `ProductosApi:BaseUrl` en `appsettings.json`. El contrato queda distribuido de manera intencional: configuración para localizar el host, servicio para conocer la ruta y componente para representar el resultado.

Cierre del punto: realiza una secuencia final con API normal: GET directo 200, Blazor lista dos productos, POST válido desde la interfaz, POST inválido desde cliente externo, consulta explícita de actualización y recuperación después de una caída temporal del proceso API. Esta secuencia reúne los aprendizajes sin añadir persistencia, Identity/JWT ni ninguna aplicación nueva.

Paso P5.1.21. Construir un resumen de los productos realmente visibles

Fundamento: 5.1.29–5.1.34. Hasta ahora la pantalla muestra cada producto del catálogo remoto. Como actividad de consolidación construiremos un pequeño resumen que dependa de la **lista que el componente tiene cargada**, no de una consulta adicional a la API.

El objetivo es comprobar que comprendemos el estado de interfaz. Añadiremos un contador de productos visibles y una suma de sus precios. Esa suma no significa un importe de ventas ni un dato persistente de SQL Server. Representa exclusivamente la colección recibida más recientemente por el componente.

Estado inicial: el proyecto Blazor B conserva `CatalogoRemoto.razor`, con consulta GET, creación POST, botón de recarga y tratamiento de errores. El servicio tipado y la API de referencia permanecen inalterados. La ampliación solo afecta a la representación Razor.

Abre:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/BLAZOR/CursoWeb.Blazor/Components/Pages/CatalogoRemoto.razor`

Busca el bloque de renderizado que contiene `<ul id="productos-api">`. Después del cierre de esa lista, pero **antes** de cerrar la rama `else` que representa un catálogo con elementos, añade:

```
<p id="resumen-catalogo-api">
    Productos visibles: @productos.Count;
    importe total mostrado: @productos.Sum(p => p.Precio).ToString("0.00") €.
</p>
```

La propiedad `Count` consulta el número de objetos de la lista local. La expresión `Sum` recorre los DTO que esa misma lista contiene y acumula sus precios. No se envía una petición HTTP nueva porque `productos` ya estaba en memoria del componente.

Por qué colocarlo dentro de esa rama: cuando hay un error o todavía está cargando, no queremos presentar un resumen como si correspondiera a una respuesta reciente. Cuando la API devuelve `[]`, el componente muestra su mensaje específico de catálogo vacío. La ampliación cuantifica el estado visible solamente cuando hay productos.

Paso P5.1.22. Comparar el código editado con una variante completa y reversible

Para evitar que un alumno reconstruya un archivo entero desde fragmentos incompletos, se ha incluido una variante completa de la página:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/VARIANTES/P5.1_RESUMEN/Components/Pages/CatalogoRemoto.razor`

Este archivo contiene **la página completa** de M05 y el párrafo de resumen ya integrado. Puedes trabajar editando manualmente la página actual con el bloque anterior o sustituir temporalmente su contenido por el archivo completo de variante.

Si eliges sustituirla, detén primero Blazor y conserva una copia de la página original. Desde la raíz del repositorio:

```
$pagina = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\Components\Pages\CatalogoRemoto.r
azor"
$variante = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.1_RESUMEN\Components\Pages\CatalogoRemoto.r
azor"
Copy-Item $pagina "$pagina.p521-original" -Force
Copy-Item $variante $pagina -Force
```

No crees otra página con la misma directiva `@page "/catalogo-remoto"` dentro del mismo proyecto: tendríamos rutas duplicadas y la práctica dejaría de partir del estado previsto. La variante se encuentra deliberadamente fuera de la carpeta compilable.

Resultado intermedio: el archivo dentro del proyecto contiene una sola implementación de `ConsultarAsync`, una sola acción de crear y un párrafo de resumen. El resto de los servicios sigue idéntico.

Paso P5.1.23. Compilar antes de atribuir cualquier fallo al servidor HTTP

La modificación se hizo dentro de un archivo Razor. La primera operación debe ser compilar Blazor, no reconstruir la API:

```
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj -c Release
```

Si la compilación falla, comprueba los delimitadores Razor, la ubicación del bloque y la sintaxis de las expresiones `Count` y `Sum`. No modifiques la ruta `api/productos` para arreglar un error sintáctico de la vista.

Después inicia la API normal y Blazor con las direcciones utilizadas en P5.1.2 y P5.1.3. Abre `http://localhost:5092/catalogo-remoto`. Deben aparecer los dos productos iniciales y el resumen correspondiente a esos elementos.

El valor numérico del resumen procede de los precios que la API entrega. No escribas a mano un total fijo para los dos productos, porque la suma debe cambiar después de una creación y de una recarga.

Paso P5.1.24. Comprobar que una creación afecta a la colección y al resumen

Con la API normal, introduce un producto de nombre Regla y precio 1,75 desde el formulario Blazor. El servicio recibe la representación creada y el componente incorpora el DTO a `productos`. El resumen debe reflejar ahora un elemento adicional y su precio sumado.

La operación no modifica el código del servicio. Si el resumen sigue mostrando el número anterior aunque el producto aparezca en la lista, revisa si el párrafo está leyendo la misma colección y si la actualización de interfaz se ha producido.

No confundas la suma de precios con la asignación del ID. El identificador lo proporciona el servidor; el resumen se calcula localmente sobre DTO ya recibidos. Son dos responsabilidades diferentes y ambas pueden verificarse sin modificar la API.

Desde otra terminal realiza GET directo a `/api/productos` y confirma que el producto existe en el proceso API. El resumen local no sustituye esa comprobación de estado remoto.

Paso P5.1.25. Demostrar que el resumen puede quedar temporalmente obsoleto

Mantén la página Blazor abierta, con el resumen visible. Desde otra terminal envía un POST válido a la API para crear un nuevo producto. No pulses todavía «Consultar servidor».

Observación esperada: el servicio remoto ha añadido el nuevo producto, pero el componente no ha recibido una notificación. Su lista y su resumen pueden seguir representando el estado anterior. No es correcto considerar la suma antigua un fallo de cálculo; es una suma correcta sobre una colección que aún no se ha actualizado.

Pulsa «Consultar servidor». La lista se reemplaza por la nueva respuesta del servidor. El contador y la suma cambian conjuntamente porque se calculan sobre los objetos que ahora están en `productos`.

Este resultado permite demostrar visualmente la idea fundamental de M05: el componente presenta una representación local de datos remotos y depende de una operación explícita para obtener otra instantánea.

Paso P5.1.26. Diferenciar un resumen vacío de un error de comunicación

Detén Blazor y cambia temporalmente la API por la variante de diagnóstico explicada en P5.1.9. Iníciala en modo `vacio` para producir HTTP 200 con un arreglo `[]`, vuelve a arrancar Blazor y abre el catálogo remoto.

La página debe mostrar el mensaje de catálogo vacío válido, no un resumen con importes inventados. El bloque que añadimos se encuentra dentro de la rama que se usa cuando existen productos; por ello no aparece en este caso.

Detén después la API y realiza una nueva consulta desde Blazor. La pantalla debe presentar el mensaje de error correspondiente, no «el importe total es cero» como si existiera una respuesta satisfactoria. Así comprobamos que el orden de renderizado sigue siendo coherente con los estados enseñados en teoría.

Una vez terminada la observación, restaura la API original y elimina la variable de escenario. No dejes activo un `Program.cs` que devuelve respuestas deliberadamente defectuosas.

Paso P5.1.27. Restaurar el componente y el servidor tras la ampliación

Si realizaste la transformación copiando la variante completa, detén el host Blazor y restaura el archivo original:

```
$pagina = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\Components\Pages\CatalogoRemoto.r
azor"
Copy-Item "$pagina.p521-original" $pagina -Force
Remove-Item "$pagina.p521-original"
```

```
dotnet build .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\BLAZOR\CursoWeb.Blazor\CursoWeb.Blazor.csproj
```

Si editaste manualmente el párrafo, puedes conservar la ampliación en tu copia de trabajo para seguir aprendiendo, pero el proyecto de referencia del repositorio permanece con el estado canónico. La variante completa permite repetir el ejercicio sin duplicar componentes dentro de la aplicación.

Vuelve a iniciar la API normal y comprueba que el catálogo responde. El ejercicio se considera finalizado cuando puedes explicar cuándo se calcula el resumen, qué información utiliza y por qué un POST de otro cliente no actualiza por sí solo la pantalla.

Paso P5.1.28. Diagnóstico final del flujo Blazor/API mediante observaciones independientes

Resume la actividad sin limitarte a decir «funciona». Debes ser capaz de establecer:

- Qué proceso ejecuta el componente interactivo.
- Qué servicio compone la solicitud HTTP.
- Qué servidor conserva los productos.
- Qué respuesta confirma una creación.
- Qué acción actualiza la representación visible.
- Por qué un contador de elementos de la pantalla puede ser distinto del número actual de recursos en la API.
- Qué diferencia una respuesta válida vacía de un error de conexión.

Estas preguntas no exigen crear otra arquitectura. Relacionan todos los archivos ya utilizados en M03, M04 y M05 y explican un caso realista de divergencia temporal entre interfaz y servidor. La respuesta satisfactoria debe mencionar las evidencias observadas en HTTP y en la pantalla, no únicamente las intenciones del código.

Resultado final y conexión con P5.2

La aplicación B está consolidada cuando GET, POST, validación, error y recarga quedan comprensibles y reproducibles. Detén ambos procesos al terminar. Desde P5.2 dejamos de utilizar Blazor y su API: los tres mecanismos de publicación siguientes se comprobarán **solo con la aplicación MVC A**, evitando ejecutar servicios adicionales que no participan en ese ensayo.

Ampliación autónoma P5.1: tres causas de una pantalla vacía

Estudia por separado estos casos sin alterar el contrato del proyecto principal: API apagada, endpoint correcto que devuelve lista vacía y endpoint que devuelve JSON incompatible. Una lista vacía válida puede representarse como `[]`; una conexión rechazada no tiene una respuesta HTTP 200; una respuesta con JSON incompatible falla durante la deserialización.

La conclusión no debe depender solo del color de un mensaje en pantalla. Explica en cuál de las capas nace cada resultado y qué comprobación directa lo demuestra. Una vez finalizado, restaura la API original de referencia para que el siguiente alumno comience desde un estado conocido.

P5.2. Compilar, ejecutar y publicar la aplicación MVC A

Objetivo y material inicial

Recuperaremos MVC A de M02. Es importante comenzar desde el proyecto correcto: la carpeta `PROYECTO/MVC/CursoWeb.Mvc` conserva controladores, vistas Razor, un ejemplo Razor Pages, EF Core 8 y SQL Server Express LocalDB. No vamos a crear una nueva plantilla MVC, porque ello rompería la continuidad de M01 y M02. Tampoco vamos a copiar archivos de Blazor ni de la API a MVC.

El estado inicial de esta actividad es el proyecto MVC con su cadena `ConnectionStrings:Productos` y el CRUD existente. El estado final será una **publicación independiente de Visual Studio**, comprobada en cuanto a archivos y preparada para ejecutarse mediante Kestrel.

Paso P5.2.1. Identificar los tres conceptos con acciones concretas

Abre una terminal PowerShell en la raíz del repositorio y define las rutas, sin copiarlas a un directorio dentro de `wwwroot`:

```
$proyecto = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj"
$salida = ".\.work\M05\publicacion-mvc"
New-Item -ItemType Directory -Force ".\.work\M05" | Out-Null
```

Primero compila:

```
dotnet build $proyecto -c Release
```

Observa el mensaje de finalización y la carpeta `bin/Release/net8.0` del proyecto. Una compilación sin errores solo demuestra que se han podido producir los ensamblados; no confirma ni que LocalDB esté accesible ni que el host haya respondido a una petición HTTP.

Paso P5.2.2. Comprobar que la aplicación sigue siendo la de M02

Abre el archivo `Program.cs`. Puedes contrastarlo con la versión acumulativa original del módulo M02, guardada fuera del proyecto compilable en `PROYECTO/VARIANTES/P5.2_ANTES/Program.cs`; esa variante es solo una referencia inicial para reconocer el añadido de M05. Localiza las dos responsabilidades que llegan del módulo anterior:

```
builder.Services.AddControllersWithViews();
builder.Services.AddRazorPages();
```

El programa debe seguir registrando el contexto mediante `UseSqlServer` con la cadena `Productos`. No lo sustituyas por SQLite ni elimines los controladores de productos. El código de M05 añade una opción explícita de despliegue que examinaremos más tarde; **por defecto en el laboratorio Windows el modo con LocalDB sigue disponible**.

Desde Visual Studio inspecciona `Views/Productos` y `Pages/Comparacion.cshtml`; comprueba que la estructura permanece. Esta observación es relevante: publicar una aplicación existente no equivale a reconstruirla desde una plantilla nueva.

Paso P5.2.3. Ejecutar el proyecto desde su origen

Con SQL Server Express LocalDB disponible en Windows y sin un proceso previo en el mismo puerto, ejecuta:

```
dotnet run --project $proyecto -c Release --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/` y `http://localhost:5091/Productos/Estado`. La segunda ruta debe devolver `Catalogo MVC disponible`. Si además abres `/Productos`, el resultado correcto dependerá de que LocalDB funcione realmente bajo tu usuario. Si falta la instancia o no se puede crear la base, no declares que el CRUD ha quedado probado: resuelve esa dependencia antes de usarlo como evidencia.

Detén el proceso con `Ctrl+C` para liberar el puerto. No mantengas al mismo tiempo dos aplicaciones MVC intentando ocupar 5091.

Paso P5.2.4. Publicar a una carpeta conocida

Ejecuta:

```
dotnet publish $proyecto -c Release -o $salida
```

Inspecciona el contenido:

```
Get-ChildItem $salida | Select-Object Name, Length
Get-ChildItem "$salida\wwwroot" -Recurse | Select-Object FullName
```

Localiza la DLL, los archivos `.deps.json` y `.runtimeconfig.json`, la configuración y los recursos estáticos. El archivo `web.config` puede aparecer como parte del soporte de publicación ASP.NET Core para IIS. La ausencia de una vista como `.cshtml` físico separado no demuestra que la vista esté perdida; habrá que abrirla después desde una petición real.

Si `dotnet publish` falla, revisa la salida completa. No borres el proyecto original para volver a empezar: una incidencia de restauración, compilación o permiso de carpeta se diagnostica por separado.

Paso P5.2.5. Comparar ejecución del proyecto y de publicación

Compara estas dos instrucciones:

```
# Ejecuta usando la estructura de proyecto:
dotnet run --project $proyecto -c Release --no-launch-profile --urls http://localhost:5091

# Ejecución de la publicación, desde su directorio:
Push-Location $salida
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
Pop-Location
```

No ejecutes las dos a la vez. La segunda es el punto de partida de P5.3 y presupone que la salida existe. Si `dotnet .\CursoWeb.Mvc.dll` falla por una dependencia LocalDB, comprueba si la base está disponible y recuerda que ese fallo de arranque no invalida por sí solo que los archivos se hayan publicado correctamente. Para el modo sin base existe una configuración explícita que utilizaremos más tarde.

Paso P5.2.6. Reconocer los artefactos imprescindibles sin confundirlos con fuentes

Relación con la teoría: 5.2.7–5.2.11. Hemos publicado la misma aplicación que desarrollamos en M02; ahora debemos explicar qué representa cada archivo sin atribuirle capacidades que no posee.

Mantén la variable `$salida` establecida en P5.2.1. No necesitas tener el servidor en ejecución para inspeccionar los archivos. Desde PowerShell, recorre el directorio:

```
Get-ChildItem -Path $salida -File |
    Sort-Object Name |
    Select-Object Name, Length

Get-ChildItem -Path (Join-Path $salida "wwwroot") -Recurse -File |
    Select-Object Name, DirectoryName
```

Localiza `CursoWeb.Mvc.dll`, `.deps.json`, `.runtimeconfig.json`, `appsettings.json` y la carpeta `wwwroot`. Busca además `web.config` si la publicación lo ha generado. No reemplaces archivos porque no conozcas todavía su función. Para aprender a leerlos, responde en orden:

¿Cuál ejecuta el host? La DLL del proyecto es un ensamblado de aplicación; se inicia con el host `dotnet` cuando el destino dispone del runtime adecuado.

¿Cuál ayuda a resolver dependencias? El archivo `.deps.json` describe componentes que participan en la carga. No contiene instrucciones para modificar `HomeController`.

¿Cuál identifica condiciones del runtime? El archivo `.runtimeconfig.json`. No instala por sí mismo un runtime faltante.

¿Dónde están los recursos estáticos? Normalmente en `wwwroot`. Revisa si existe `css/site.css`; no basta con ver que la carpeta está creada.

¿Dónde se definen valores para la ejecución? En archivos de configuración y otras fuentes como las variables de entorno. La configuración de LocalDB no instala ni copia SQL Server.

¿Para qué sirve `web.config` si existe? Para colaborar con el hospedaje IIS mediante el mecanismo ASP.NET Core correspondiente. No convierte la publicación por sí sola en un sitio IIS.

El resultado de este apartado no es un archivo nuevo. Es la capacidad de señalar la pieza correcta cuando una publicación falla por una causa determinada. Guarda la salida sin editar: la utilizaremos más adelante para ejecutar el host.

Paso P5.2.7. Una comparación reproducible entre dos versiones publicadas

Relación con la teoría: 5.2.7 y 5.2.9. Si editamos una vista del proyecto, la publicación anterior no tiene por qué incorporar el cambio. Para comprobarlo construiremos dos salidas separadas de **la misma aplicación MVC**, cambiando una sola vista Razor entre una publicación y la siguiente.

Este paso es una ampliación de consolidación que puede realizarse después de la jornada. No crea otra aplicación ni modifica el resto del curso. La variante de referencia está guardada como archivo Razor **completo** en:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/VARIANTES/P5.2_CAMBIO_VISTA/Views/Home/Index.cshtml`

El archivo contiene el contenido original de `Views/Home/Index.cshtml` y añade un único párrafo identificable. Para reconstruir manualmente el cambio, abre el archivo del proyecto MVC y, debajo del elemento con identificador `resultado-inicial`, introduce:

```
@* AMPLIACIÓN P5.2: marca visible solo en la segunda publicación. *@
<p id="marca-publicacion-m05">
    Esta versión se publicó después de modificar la vista MVC.
</p>
```

Estado A — publicación original. Detén cualquier proceso MVC que esté utilizando la carpeta de salida. Desde la raíz del repositorio prepara rutas de trabajo que no se versionan:

```
$proyecto = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj"
$vista = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\Views\Home\Index.cshtml"
$variante = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.2_CAMBIO_VISTA\Views\Home\Index.cshtml"
$respaldo = "$vista.p52-original"
$publicacionA = ".\.work\M05\publicacion-original"
$publicacionB = ".\.work\M05\publicacion-cambio"

Copy-Item $vista $respaldo -Force
dotnet publish $proyecto -c Release -o $publicacionA
```

Qué se espera: `publicacion-original` contiene el estado de la vista antes de añadir la marca P5.2, porque se ha publicado desde el archivo de partida. No hace falta copiar ningún `.cshtml` manualmente a la carpeta de salida: el SDK prepara los artefactos de la aplicación.

Estado B — modificación de vista y publicación posterior. Después de conservar el estado A, cambia el origen:

```
Copy-Item $variante $vista -Force
dotnet publish $proyecto -c Release -o $publicacionB
```

Si prefieres escribir la modificación manualmente en vez de copiar la variante, el resultado del archivo debe ser el mismo, sin duplicar el elemento `marca-publicacion-m05`. Para comprobarlo, abre la vista y verifica que ese identificador aparece **una sola vez**.

Hasta aquí hemos producido dos directorios publicados, pero todavía no se ha probado ninguno por HTTP. Eso es intencional: separa lo que hace `dotnet publish` de la ejecución.

Paso P5.2.8. Ejecutar ambas publicaciones y localizar el cambio por HTTP

Para esta comparación prescindiremos de la inicialización LocalDB porque solo necesitamos la portada MVC. En una primera terminal PowerShell, desde la raíz del repositorio:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location .\.work\M05\publicacion-original
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

Mantén ese proceso iniciado. Abre una segunda terminal independiente en la raíz del repositorio:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location .\.work\M05\publicacion-cambio
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5094
```

Ambas instancias utilizan puertos diferentes. La primera debe continuar sirviendo la portada de la publicación original; la segunda debe representar la vista nueva. En una tercera terminal o con dos pestañas del navegador compara:

```
curl.exe -i http://localhost:5091/
curl.exe -i http://localhost:5094/
```

En la segunda respuesta debe aparecer el texto «Esta versión se publicó después de modificar la vista MVC» dentro del elemento `marca-publicacion-m05`. En la primera no debe aparecer. No basta con obtener 200 en ambas: la evidencia buscada es que **las dos publicaciones contienen versiones distintas de una misma vista del mismo proyecto**.

Repite la consulta a `http://localhost:5091/` después de haber publicado B. Debe seguir mostrando la vista original porque ese proceso utiliza los artefactos A. Así comprobamos que publicar a una carpeta diferente no modifica retroactivamente una publicación anterior.

Si ambas páginas muestran la marca: comprueba que la publicación A se produjo antes de sustituir la vista y que el proceso 5091 no está ejecutando archivos de la carpeta B. **Si ninguna la muestra:** comprueba que realmente publicaste después de modificar la fuente y que 5094 está sirviendo la salida B. El diagnóstico debe relacionar origen, destino y proceso; no volver a crear un proyecto MVC.

Paso P5.2.9. Restaurar el estado del curso sin destruir las publicaciones de ensayo

Detén los dos procesos con Ctrl+C en sus respectivas terminales y ejecuta `Pop-Location` en ambas. Después, desde la raíz del repositorio, restaura el archivo de origen:

```
$vista = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\Views\Home\Index.cshtml"
$respaldo = "$vista.p52-original"

Copy-Item $respaldo $vista -Force
Remove-Item $respaldo
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue

dotnet build .\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj -c
Release
```

Las carpetas de publicación A y B permanecen dentro de `.work` para facilitar una comparación posterior y no pertenecen al material que versiona el alumno. Si quieres repetir el ensayo, vuelve a publicar desde un estado inicial conocido. No modifiques `Views/Home/Index.cshtml` de M02 ni borres su trabajo previo.

Conclusión razonada de este ejercicio: no se trata de aprender dos comandos, sino de comprobar que el código fuente, la salida A, la salida B y los dos procesos HTTP son objetos distintos. La publicación nueva solo se manifiesta cuando la ejecución utiliza los artefactos nuevos.

Paso P5.2.10. Publicar por primera vez desde una ruta absoluta controlada

Fundamento: 5.2.21 y 5.2.24. Hasta ahora hemos utilizado rutas relativas desde la raíz del repositorio. Este ejercicio permite comprobar qué sucede cuando se trabaja desde otra carpeta y por qué conviene conocer la ubicación real de los artefactos.

Abre PowerShell en la raíz del repositorio. Localiza el archivo de proyecto e inspecciona su ruta absoluta:

```
$proyecto = Resolve-Path ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj"
$proyecto.Path
```

No copies el resultado literal a otro equipo. La ubicación dependerá de dónde haya clonado el alumno el repositorio. Crea un destino también absoluto, pero dentro de la zona de trabajo de ese repositorio:

```
New-Item -ItemType Directory -Force ".\.work\M05" | Out-Null
$carpeta = (Resolve-Path ".\.work\M05").Path
$destino = Join-Path $carpeta "publicacion-ruta-absoluta"

dotnet publish $proyecto.Path -c Release -o $destino
```

Cuando termine, comprueba:

```
Test-Path (Join-Path $destino "CursoWeb.Mvc.dll")
Test-Path (Join-Path $destino "CursoWeb.Mvc.runtimeconfig.json")
Test-Path (Join-Path $destino "wwwroot\css\site.css")
```

Los tres resultados deberían ser verdaderos. Explica qué confirma cada uno y qué no confirma todavía. El primer resultado localiza el ensamblado; el segundo, un archivo que describe requisitos de ejecución; el tercero, un recurso estático. Ninguno constituye por sí solo una petición HTTP atendida.

Cambia temporalmente el directorio actual con `Push-Location` a una carpeta diferente y vuelve a ejecutar la misma orden `dotnet publish` usando `$proyecto.Path` y `$destino`. Como ahora ambas variables contienen rutas absolutas, la operación sigue apuntando a los mismos orígenes y destino. Ejecuta `Pop-Location` para regresar.

La intención no es recomendar que todos los comandos se escriban siempre con rutas largas; es enseñar que **el directorio actual tiene consecuencias**, y que el alumno puede eliminarlas cuando necesite una reproducción exacta.

Paso P5.2.11. Examinar la salida de publicación sin borrar archivos desconocidos

Fundamento: 5.2.8 y 5.2.22. Un error común durante el aprendizaje es eliminar archivos publicados cuyo nombre parece misterioso. Antes de hacerlo debemos comprender su función.

Usa el destino que acabas de generar:

```
Get-ChildItem -Path $destino -File |
    Select-Object Name, Extension, Length |
    Sort-Object Extension, Name
```

Localiza la DLL principal y los archivos `.deps.json` y `.runtimeconfig.json`. Después identifica si existen archivos de configuración y una carpeta `wwwroot`.

Abre el archivo de configuración del runtime con un editor de texto. No modifiques su contenido; busca la referencia al framework exigido. Observa que el archivo no instala ese framework: describe la versión y las condiciones de ejecución esperadas.

Abre también `appsettings.json`. Si contiene `ConnectionStrings:Productos`, no debes interpretar que la base física haya sido copiada. El archivo contiene un valor de configuración; la instancia de SQL Server Express LocalDB pertenece al entorno Windows.

Finalmente comprueba el archivo estático:

```
Get-Item (Join-Path $destino "wwwroot\css\site.css") |
    Select-Object Name, Length, LastWriteTime
```

Si existe y tiene contenido, sabemos que se ha preparado un recurso estático. Todavía no sabemos si el middleware lo va a servir, porque eso exige iniciar la aplicación y pedir su URL. Deja intactos los archivos. No es necesario editar manualmente `.deps.json` o `.runtimeconfig.json` para enseñar un despliegue elemental.

Paso P5.2.12. Comprobar los runtimes disponibles antes de iniciar la publicación

Fundamento: 5.2.17. Desde PowerShell ejecuta:

```
dotnet --list-sdks
dotnet --list-runtimes
```

El primer comando informa de los SDK instalados; el segundo, de los runtimes. Busca una versión compatible de Microsoft.AspNetCore.App 8 en la lista de runtimes. La publicación que utilizamos depende del framework y presupone que el entorno puede ejecutar ASP.NET Core 8.

El alumno debe explicar por qué **tener el SDK no es la única condición de ejecución de un destino**, y por qué una carpeta publicada no garantiza que un equipo remoto tenga los runtimes necesarios. En su puesto de desarrollo quizá aparezcan tanto SDK como runtime porque Visual Studio los instaló; en un servidor con IIS el escenario habitual será diferente.

No desinstales ni cambies runtimes como parte de esta práctica. El objetivo es reconocer sus requisitos, no provocar una avería que impida continuar el curso. Si falta .NET 8 en el puesto del alumno, hay que preparar el laboratorio antes de ejecutar las actividades.

Paso P5.2.13. Contrastar tres evidencias de una publicación ejecutable

Fundamento: 5.2.19. Inicia la DLL de la carpeta publicada en modo sin base para estudiar la parte web del host, sin alterar EF Core:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location $destino
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5094
```

Mantén esa ventana abierta. Desde otra terminal ejecuta:

```
curl.exe -i http://localhost:5094/
curl.exe -i http://localhost:5094/Productos/Estado
curl.exe -i http://localhost:5094/css/site.css
```

Anota tres observaciones: la portada HTML se genera, la acción de estado responde el texto previsto y el recurso estático se sirve desde la publicación. Esto demuestra más que la mera presencia de archivos. El siguiente paso lógico sería comprobar el CRUD con SQL Server en un escenario donde la base sea accesible, pero no lo confundas con el ensayo sin base.

Detén el proceso con Ctrl+C, ejecuta `Pop-Location` y elimina la variable en esa terminal:

```
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue
```

El recorrido conserva la salida dentro de `.work`, por lo que puede repetirse sin reconstruir el proyecto. No hace falta modificar ningún archivo fuente.

Paso P5.2.14. Conservar el proveedor SQL Server cuando el destino no ofrece LocalDB

Fundamento: 5.2.12 y 5.2.29. Abre `PROYECTO/MVC/CursoWeb.Mvc/Program.cs` y localiza el registro del contexto con `UseSqlServer`. Comprueba que la aplicación sigue utilizando la cadena `ConnectionStrings:Productos` como en M02.

A continuación encuentra la lectura de `Despliegue:SinLocalDb` y el bloque que decide si inicializar la base. Identifica una relación de causa y efecto: cuando la opción es falsa, el programa intenta preparar LocalDB. Cuando es verdadera, omite ese arranque, pero no reemplaza SQL Server por una colección estática.

Localiza finalmente el middleware que intercepta las rutas bajo `/Productos`, exceptuando `/Productos/Estado`, para devolver 503 en el modo sin base. Esta parte es tan importante como la omisión de inicialización: evita que el alumno crea que un CRUD funciona por el mero hecho de que la portada abre.

Comparación razonada:

Situación	Interpretación admisible
`dotnet publish` termina correctamente	Artefactos generados
Kestrel responde 200 en `/`	El host y la portada funcionan
Kestrel responde 503 en `/Productos` con modo sin base	El CRUD está explícitamente deshabilitado
`/Productos` funciona con LocalDB real en Windows	Se ha comprobado, al menos, una ruta SQL
Crear, editar y eliminar productos con LocalDB	Evidencia de recorrido CRUD persistente

No cambies el proveedor a SQLite para conseguir una respuesta 200 en el contenedor posterior. Eso violaría el contrato tecnológico del curso y alteraría el ejemplo que el alumno ha construido desde M02.

Paso P5.2.15. Construir dos salidas y demostrar que el navegador consulta dos versiones

Fundamento: 5.2.13–5.2.15 y 5.2.25. Recupera las carpetas `publicacion-original` y `publicacion-cambio` de P5.2.7. Ambas pertenecen al mismo proyecto, pero la segunda se generó después de introducir un marcador en `Views/Home/Index.cshtml`.

Inicia A y B en ventanas diferentes, con la opción sin base y puertos 5091 y 5094 respectivamente. En el navegador abre ambos documentos y examina el texto. Para identificar el cambio sin depender exclusivamente del aspecto visual, usa desde otra terminal:

```
curl.exe -s http://localhost:5091/ | Select-String "marca-publicacion-m05"
curl.exe -s http://localhost:5094/ | Select-String "marca-publicacion-m05"
```

La primera búsqueda no debería encontrar ese identificador; la segunda sí. Si ambos resultados son iguales, revisa de qué carpeta procede cada proceso y si la versión B se publicó realmente después de modificar la vista.

No necesitas crear dos proyectos ni dos controladores. Las versiones del programa se distinguen por su contenido publicado y por el proceso que lo utiliza. Detén ambas instancias y confirma que la fuente se ha restaurado según P5.2.9.

Paso P5.2.16. Ejercicio de diagnóstico sin modificar el proyecto

Un compañero afirma que «publicar no funciona» porque abrió el navegador después de ejecutar `dotnet publish` y no encontró ninguna página. Antes de corregir su equipo, hazle estas preguntas:

- ¿Dónde se encuentra el directorio de salida?
- ¿Se ejecutó realmente una DLL publicada o solamente se generaron los archivos?
- ¿Qué proceso debería escuchar la URL indicada?
- ¿Existe una dirección de escucha anunciada por ese proceso?
- ¿Qué respuesta concreta se obtiene al solicitar una ruta conocida?

Ahora considera un segundo compañero que obtiene 200 en la portada y 503 en Productos. ¿Se ha demostrado que la publicación es defectuosa? No necesariamente. Puede estar utilizando la opción sin LocalDB que limita expresamente esas rutas. La comprobación de la configuración es más pertinente que rehacer la vista.

Por último, otro compañero abre una portada antigua después de volver a publicar. Debe comprobar el proceso activo y la carpeta que utiliza antes de cambiar código C#. La respuesta correcta identifica **qué instancia está recibiendo la solicitud**.

Estas tres situaciones son una evaluación formativa completa del concepto publicar: salida, ejecución y dependencia. No requieren una herramienta adicional; requieren interpretar evidencias diferentes.

Paso P5.2.17. Publicar MVC A desde Visual Studio sin cambiar de proyecto

Fundamento: 5.2.21–5.2.24. Hemos utilizado `dotnet publish` desde PowerShell. Ahora reproduciremos el mismo concepto mediante Visual Studio, para que el alumno reconozca la correspondencia entre la experiencia gráfica del entorno y los artefactos de publicación.

Abre en Visual Studio la solución o el proyecto que corresponde a MVC A dentro de `M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/MVC/CursoWeb.Mvc`. No abras accidentalmente la API auxiliar ni Blazor B: esos proyectos pertenecen al recorrido P5.1 y no son el objeto de despliegue de P5.2–P5.5.

En el Explorador de soluciones selecciona **el proyecto web MVC**, no el nodo de la solución si aparecen otros proyectos. Utiliza la opción «Publicar» del menú contextual del proyecto. Crea un

perfil nuevo y elige como destino una **carpeta**. No selecciones Azure, un contenedor ni IIS en esta fase, porque el objetivo es generar primero los archivos publicados para inspeccionarlos.

Indica una carpeta de ensayo identificable, por ejemplo una subcarpeta de `.work\M05\publicacion-vs`, y elige una configuración de publicación Release dependiente del framework cuando el asistente la permita. Conserva las opciones de plataforma compatibles con .NET 8; no conviertas el proyecto a otra versión del framework.

El asistente puede variar entre versiones de Visual Studio. Lo que importa no es memorizar la posición exacta de cada botón, sino registrar el **tipo de destino** y la **ruta física** que seleccionaste. Si una pantalla de tu versión utiliza otra denominación, no aceptes un destino distinto por su parecido visual: confirma que el resultado será una publicación a carpeta local.

Paso P5.2.18. Comparar la salida gráfica con la salida de `dotnet publish`

Cuando Visual Studio informe de que la publicación terminó, abre PowerShell desde la raíz del repositorio. Localiza la carpeta que elegiste en el asistente. Si utilizaste la ruta de referencia, prepara una variable:

```
$publicacionVs = ".\.work\M05\publicacion-vs"
Get-ChildItem $publicacionVs | Select-Object Name, Length
```

Comprueba que el directorio contiene `CursoWeb.Mvc.dll`, `.deps.json`, `.runtimeconfig.json`, archivos de configuración y recursos de `wwwroot`. No supongas que porque se utilizó la interfaz gráfica el resultado dejó de ser una aplicación .NET ejecutable.

Compara esa salida con la generada previamente desde consola:

```
$publicacionCli = ".\.work\M05\publicacion-mvc"
Get-ChildItem $publicacionCli | Select-Object Name, Length
```

Las carpetas pueden contener diferencias de fecha o de artefactos dependientes del perfil seleccionado. No es necesario que todos los tamaños de archivo sean idénticos para aprender el concepto. Lo importante es que ambas salidas corresponden a una publicación del proyecto web MVC correcto y contienen los recursos necesarios.

Si Visual Studio genera una publicación en otra carpeta distinta de la prevista, revisa primero el perfil y la ruta seleccionada. No copies manualmente archivos sueltos desde `bin` a un directorio cualquiera para ocultar la discrepancia.

Paso P5.2.19. Ejecutar la publicación creada desde Visual Studio

Fundamento: 5.2.22. La interfaz gráfica ha producido una carpeta. Todavía no existe necesariamente ningún servidor HTTP atendiendo su contenido. Vamos a iniciar la DLL desde la salida gráfica exactamente como hicimos con la CLI.

Desde PowerShell, con cualquier instancia previa de MVC detenida:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location ".\.work\M05\publicacion-vs"
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5094
```

Abre desde el navegador la dirección `http://localhost:5094/`. Después solicita Estado y el CSS publicado. El modo sin base permite concentrarse en el alojamiento sin necesitar una instancia LocalDB bajo una identidad diferente.

Observa que la aplicación no necesita ejecutarse desde Visual Studio una vez publicada. La salida creada por su asistente tiene las mismas responsabilidades de ejecución que la salida producida mediante `dotnet publish`.

Detén el proceso con Ctrl+C, regresa con `Pop-Location` y elimina la variable temporal en esa PowerShell. Si tienes otro proceso MVC abierto en el mismo puerto, libéralo antes de interpretar el resultado de la prueba.

Paso P5.2.20. Interpretar una publicación creada en el directorio incorrecto

Un alumno cree haber publicado a `.work\M05\publicacion-vs`, pero ese directorio no existe. Visual Studio informó de éxito. ¿Qué paso debe comprobarse primero?

La solución no es cambiar el controlador MVC. Debe revisar el perfil de publicación y la ruta física que figuraba como destino. Una ruta relativa puede haberse interpretado desde el directorio del proyecto o del perfil, por lo que el resultado puede existir en otra ubicación.

Revisar el destino real es más útil que repetir el botón «Publicar» varias veces sin saber dónde se escribirán los artefactos. Cuando se identifica la carpeta correcta, se puede inspeccionar la DLL y los recursos y ejecutar la publicación desde ese directorio.

Este caso es una adaptación del mismo principio enseñado con la CLI: el resultado de la operación depende tanto del proyecto seleccionado como del destino. El editor o la terminal no alteran esa necesidad de precisión.

Paso P5.2.21. Evitar que el perfil de Visual Studio se confunda con un sitio IIS

Un perfil de publicación a carpeta no crea automáticamente un sitio en Internet Information Services. Genera una salida que después podrá alojarse de distintas maneras. En el siguiente punto esa salida se ejecuta con Kestrel; en P5.4 se coloca en un directorio físico utilizado por IIS; y en P5.5 Docker construye una imagen a partir del proyecto.

Si el alumno elige un perfil de despliegue directo a IIS durante P5.2, habrá mezclado dos conceptos antes de aprenderlos. La secuencia acordada separa las fases para poder comprobar qué significa cada operación.

El perfil tampoco crea automáticamente una instancia SQL Server Express LocalDB en otro equipo. La aplicación conserva su cadena de conexión, pero las condiciones de acceso a la base siguen dependiendo de la identidad y del entorno de ejecución. Esta distinción debe mantenerse cuando se estudie el Application Pool.

Paso P5.2.22. Comprobar que Visual Studio publicó MVC A y no Blazor B

En cursos con varios proyectos web resulta fácil publicar el proyecto seleccionado por error. Para identificar MVC A, busca en la salida `CursoWeb.Mvc.dll` y solicita una ruta del recorrido MVC, como `/Productos/Estado`.

Una salida de Blazor B tendrá otros ensamblados, componentes y rutas. Aunque ambas aplicaciones utilicen ASP.NET Core, no son intercambiables. No serviremos el catálogo remoto de Blazor para demostrar el hospedaje de la aplicación MVC A.

Pregunta razonada: ¿por qué no basta con comprobar que la carpeta publicada contiene algún archivo `.dll`? Porque cualquier proyecto .NET puede producir un ensamblado. Necesitamos vincularlo a la aplicación y a las rutas concretas cuyo despliegue estamos ensayando.

La comprobación del nombre de proyecto, de DLL y de ruta funcional permite corregir una publicación equivocada antes de comenzar IIS o Docker.

Paso P5.2.23. Distinguir una publicación antigua de una publicación nueva en Visual Studio

Supongamos que el docente modifica el texto de Home/Index, vuelve a publicar desde Visual Studio y sigue viendo la portada anterior en un proceso Kestrel. Hay dos preguntas que deben responderse en orden.

Primero, ¿en qué carpeta dejó Visual Studio la nueva publicación? Comprueba el destino real del perfil. Segundo, ¿qué directorio utiliza el proceso que está respondiendo la URL del navegador? Si continúa ejecutando la carpeta anterior, el contenido visible puede mantenerse antiguo aunque el asistente haya generado archivos nuevos.

No se debe atribuir automáticamente el problema a la caché del navegador, a Entity Framework Core o a los controladores. La relación entre fuente, artefacto y proceso proporciona un diagnóstico más directo.

La práctica P5.2.7 permitió comparar dos versiones con un marcador visible. Puedes utilizar esa experiencia para explicar el caso sin volver a modificar el proyecto.

Paso P5.2.24. Comparar dos interfaces para una misma operación técnica

La publicación gráfica y la de terminal pueden producir artefactos compatibles para la misma aplicación. La diferencia principal es cómo se expresan las opciones y cómo se observa el resultado.

En PowerShell elegimos proyecto, configuración y salida con argumentos como `-c Release` y `-o`. En Visual Studio elegimos el proyecto y un perfil de publicación a carpeta, y revisamos las opciones del asistente. En ambos casos debemos conocer el destino y el framework objetivo, y después ejecutar una comprobación HTTP independiente si queremos afirmar que la aplicación funciona.

El alumnado debe poder explicar qué gana con la interfaz gráfica y qué aporta la CLI. No es necesario elegir una como universalmente superior. La primera facilita descubrir opciones con guía visual; la segunda permite reproducir una operación con comandos precisos. Ambas requieren comprender qué archivos se generan.

Paso P5.2.25. Consolidación del proceso antes del hospedaje real

Termina esta ampliación asegurándote de que las carpetas publicadas no se han mezclado con el código fuente. Conserva las salidas de ensayo bajo `.work\M05`, detén los servidores temporales y restaura las variables de entorno utilizadas en las pruebas.

El resultado que debes poder explicar es concreto: un proyecto MVC A fue compilado y publicado tanto desde terminal como desde Visual Studio; los artefactos se inspeccionaron y una DLL publicada atendió rutas HTTP mediante Kestrel.

Ninguna de esas acciones sustituyó el proveedor EF Core SQL Server ni convirtió el proyecto en una aplicación diferente. El siguiente paso es estudiar otros mecanismos de alojamiento manteniendo constante la misma publicación MVC.

Paso P5.2.26. Comparar dos fuentes de configuración sin recompilar MVC

Fundamento teórico: 5.2.4, 5.2.10 y 5.2.12. En los experimentos anteriores activamos el modo sin base mediante una variable de entorno de PowerShell. Ahora obtendremos el mismo comportamiento a partir de un archivo específico del entorno **Production**, pero solamente dentro de la carpeta publicada.

Estado inicial: disponemos de la publicación MVC A en `.work\M05\publicacion-mvc`, con el proyecto fuente intacto. No existe ningún proceso que esté ejecutando esa carpeta en el puerto 5091. Para evitar sobrescrituras de archivos que podrían pertenecer a otro ensayo, antes de comenzar comprueba que no existe `appsettings.Production.json` en la publicación:

```
$salida = ".\.work\M05\publicacion-mvc"
$config = Join-Path $salida "appsettings.Production.json"
if (Test-Path $config) {
    Write-Host "Ya existe configuración Production: revísala y no la sobrescribas."
}
```

Si existe, detén aquí este ensayo y consulta su contenido con el docente. No borres un archivo de configuración sin conocer su procedencia. Si la carpeta de salida todavía no se ha creado, vuelve al comando `dotnet publish` de P5.2.4.

Paso P5.2.27. Preparar el archivo JSON de la variante

El repositorio incluye un archivo íntegro de configuración docente:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/VARIANTES/P5.2_CONFIGURACION/appsettings.Production.json`

Su contenido completo es:

```
{
  "Despliegue": {
    "SinLocalDb": true
  }
}
```

No cambia el proveedor EF Core, no contiene otra cadena de conexión y no activa un almacén de productos ficticio. Expresa exclusivamente la decisión de omitir LocalDB para comprobar páginas que no dependen de SQL Server.

Copia el archivo **a la publicación**, no al árbol fuente de MVC A:

```
$variante = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.2_CONFIGURACION\appsettings.Production.json
"
Copy-Item $variante $config
Get-Content $config
```

La salida debe mostrar la clave `Despliegue:SinLocalDb`. El directorio de código fuente sigue conservando su configuración original de SQL Server Express LocalDB.

Paso P5.2.28. Ejecutar con Production sin definir la variable de despliegue

Abre una terminal PowerShell nueva y ejecuta desde la raíz del repositorio:

```
$salida = ".\.work\M05\publicacion-mvc"
$env:ASPNETCORE_ENVIRONMENT = "Production"
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue
Push-Location $salida
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

El proceso debe leer su configuración habitual y el archivo específico del entorno Production de la carpeta publicada. Si la publicación y el runtime están preparados, debería arrancar sin intentar inicializar LocalDB.

Desde otra terminal comprueba:

```
curl.exe -i http://localhost:5091/
curl.exe -i http://localhost:5091/Productos/Estado
curl.exe -i http://localhost:5091/Productos
```

La portada y Estado deben responder 200. La ruta Productos debe devolver 503. Ese comportamiento permite inferir que el modo sin base está aplicado; **no demuestra que una consulta a SQL Server se haya realizado correctamente**.

Paso P5.2.29. Repetir el resultado con una variable de entorno en vez del JSON

Detén el proceso anterior con Ctrl+C y ejecuta `Pop-Location`. Desde la raíz, elimina únicamente el archivo de variante que acabas de colocar en la publicación:

```
$config = ".\.work\M05\publicacion-mvc\appsettings.Production.json"
Remove-Item $config
```

Ahora define la opción de despliegue directamente para el nuevo proceso:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location ".\.work\M05\publicacion-mvc"
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

Solicita las mismas tres rutas. Deben producir respuestas coherentes con el modo sin base. El archivo Production ya no está presente, pero el programa recibe la opción mediante otro proveedor de configuración.

Conclusión causal: la aplicación lee una clave lógica cuyo valor puede provenir de diferentes fuentes. La configuración no es código nuevo. El mismo ensamblado puede comportarse de forma distinta al cambiar las condiciones de inicio.

Paso P5.2.30. Restaurar la carpeta y la terminal del ejercicio

Detén Kestrel, ejecuta `Pop-Location` y elimina las variables que modificaste en esa terminal:

```
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue
Remove-Item Env:\ASPNETCORE_ENVIRONMENT -ErrorAction SilentlyContinue
```

Comprueba que no quedó el `appsettings.Production.json` del ensayo dentro de `.work\M05\publicacion-mvc`. El proyecto MVC A conserva el `Program.cs` y la conexión SQL Server Express LocalDB originales.

Si el archivo específico del entorno ya existía antes de comenzar, no deberías haber realizado este recorrido sobre él. Utiliza una carpeta publicada independiente para repetir el experimento sin sobrescribir configuraciones que no te pertenecen.

El resultado que debe poder explicar el alumno es la diferencia entre **configuración incorporada a la carpeta publicada** y **variable de entorno suministrada al proceso**. Ambas pueden activar una opción del mismo código, pero no deben confundirse con compilar, publicar o proporcionar un servidor SQL.

Resultado y estado final P5.2

La carpeta `.work\M05\publicacion-mvc` contiene los artefactos de MVC. El alumno sabe identificar qué cambió al publicar: no el código fuente ni las acciones, sino los archivos preparados para ejecutarse fuera de la estructura de desarrollo. No edites la fuente para corregir un problema que solo aparece por una variable del entorno de ejecución.

Ampliación autónoma P5.2: inspección responsable de archivos

Comprueba la presencia y finalidad de seis elementos: DLL, deps, runtimeconfig, appsettings, wwwroot y web.config cuando exista. Explica por qué no debemos copiar manualmente una carpeta `obj` al servidor. Compara el tamaño de la publicación con la carpeta de fuentes, pero no infieras que el tamaño demuestra calidad. Después restaura un único directorio de salida; no produzcas múltiples carpetas «definitivas» difíciles de distinguir.

P5.3. Ejecutar la aplicación MVC publicada en Kestrel

Objetivo y estado inicial

La carpeta publicada existe desde P5.2. Ahora vamos a iniciar su aplicación **desde la DLL publicada**, verificar el puerto, obtener respuesta HTML, comprobar recursos y detener el proceso. El objetivo no es investigar nuevos controladores ni aprender una tecnología de persistencia, sino relacionar el proceso Kestrel con las peticiones que recibe.

Paso P5.3.1. Preparar el directorio y liberar el puerto

Comprueba si el puerto ya está ocupado:

```
Get-NetTCPConnection -LocalPort 5091 -ErrorAction SilentlyContinue
```

Si ves un listener de un proceso anterior que controlas, detén ese proceso antes de iniciar otro. Si el puerto no está libre o pertenece a una aplicación que no debes cerrar, elige un puerto

alternativo como 5094 y mantén ese número coherente al ejecutar y consultar el navegador. No resuelvas un conflicto de puerto editando vistas Razor.

Accede a la publicación:

```
$salida = ".\.work\M05\publicacion-mvc"
Push-Location $salida
```

Paso P5.3.2. Iniciar la publicación en el puerto definido

En una terminal interactiva:

```
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

Debes observar mensajes de arranque y una dirección de escucha, siempre que el runtime y la configuración sean válidos. No cierres la terminal mientras navegas: es la que mantiene el proceso. Desde otra terminal o el navegador solicita:

```
curl.exe -i http://localhost:5091/
curl.exe -i http://localhost:5091/Productos/Estado
curl.exe -i http://localhost:5091/css/site.css
```

Los resultados deben indicar una respuesta HTTP útil de la misma aplicación MVC. La ruta de estado responde desde un controlador sin consultar la tabla de productos. La hoja CSS permite observar que `UseStaticFiles` sigue sirviendo recursos publicados.

Paso P5.3.3. Diferenciar URL inicial y funcionamiento del CRUD

Si LocalDB se encuentra disponible, comprueba además `http://localhost:5091/Productos` y, si el instructor ha preparado los datos, la consulta de un producto. Si no se puede acceder a LocalDB, registra esa limitación como prueba de persistencia **no realizada**. La prueba 200 de Home y Estado no equivale a haber probado la creación y edición de productos.

El experimento de despliegue de este módulo permite una variante explícita sin base:

```
# Ejecutar SOLO si el objetivo es verificar el hospedaje sin SQL Server:
$env:Despliegue__SinLocalDb = "true"
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

En ese modo, `/`, `/Productos/Estado` y los recursos estáticos son la demostración de hospedaje. Las rutas CRUD dependientes de SQL Server devuelven **503 Service Unavailable** por una regla deliberada del laboratorio. No presentaremos una respuesta 503 como una consulta de productos correcta.

La opción está en el proyecto M05 y mantiene intacto el proveedor `Microsoft.EntityFrameworkCore.SqlServer`. Cuando terminemos esta variante, debemos restaurar la variable:

```
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue
```

No confundas un dato configurado en tu terminal con un cambio permanente de los archivos del proyecto.

Paso P5.3.4. Comprobar una colisión de puertos de forma controlada

Con la publicación escuchando correctamente en 5091, abre **una tercera terminal** y trata de iniciar otra instancia utilizando el mismo puerto. Si LocalDB prepara correctamente el arranque de ambas instancias, la segunda debería informar de una dirección ya utilizada. Si una de las instancias falla antes por LocalDB, registra esa causa en lugar de adjudicarle falsamente una colisión.

Para realizar este experimento de forma aislada, usa la opción `Despliegue__SinLocalDb=true` en ambas terminales y comprueba el mensaje de puerto ocupado. Detén la segunda instancia y conserva la primera hasta finalizar las comprobaciones. El estado final del laboratorio no requiere mantener dos servidores.

Paso P5.3.5. Detener, comprobar y volver a la raíz

Detén el proceso principal con `Ctrl+C`. Comprueba que la dirección deja de responder cuando realmente no exista otro proceso que la esté atendiendo. Después ejecuta:

```
Pop-Location
```

El directorio de publicación sigue existiendo. No borres la salida: será reutilizada en P5.4 como fuente de artefactos de despliegue. El estado final de P5.3 consiste en una aplicación MVC publicada capaz de servir HTML/CSS/estado bajo Kestrel y un puerto liberado.

Paso P5.3.6. Construir una prueba HTTP mínima que distinga conexión y resultado

Relación con la teoría: 5.3.8 y 5.3.12. En este ejercicio no cambiaremos el código MVC. Utilizaremos la DLL publicada y una configuración explícita para ver que un mismo servidor puede responder 200 en una ruta y 503 en otra.

Si terminaste P5.3.5, vuelve a la raíz del repositorio y abre una terminal PowerShell. Elimina una variable anterior para que la demostración no herede un valor desconocido; después elige expresamente el modo sin base:

```
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue
$env:Despliegue__SinLocalDb = "true"
$salida = ".\.work\M05\publicacion-mvc"

Push-Location $salida
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

Con la ventana abierta, desde otra terminal solicita en este orden:

```
curl.exe -i http://localhost:5091/
curl.exe -i http://localhost:5091/Productos/Estado
curl.exe -i http://localhost:5091/Productos
curl.exe -i http://localhost:5091/css/site.css
```

Anota la combinación de código HTTP y recurso solicitado. El resultado previsto es 200 en la portada, 200 en Estado, 503 en Productos y 200 en el CSS. El 503 no quiere decir que Kestrel esté detenido: **el servidor ha devuelto expresamente una respuesta** en la ruta dependiente de base. El resultado documenta una capacidad limitada del proyecto para este escenario.

Para comprobar el texto del endpoint de estado independientemente de la representación visual, utiliza:

```
curl.exe -s http://localhost:5091/Productos/Estado
```

Debes observar `Catalogo MVC disponible`. Una respuesta de texto desde una acción concreta acredita el recorrido hasta el controlador. No acredita que las consultas EF Core se hayan ejecutado.

Paso P5.3.7. Observar que un puerto pertenece a un proceso

Relación con la teoría: 5.3.7, 5.3.10 y 5.3.11. Mientras la aplicación permanece activa en 5091, abre otra terminal PowerShell:

```
Get-NetTCPConnection -LocalPort 5091 -State Listen -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess
```

Si el comando ofrece el identificador del proceso, puedes consultarlo sin detenerlo:

```
$conexiones = Get-NetTCPConnection -LocalPort 5091 -State Listen -ErrorAction SilentlyContinue
$conexiones | ForEach-Object {
    Get-Process -Id $_.OwningProcess -ErrorAction SilentlyContinue |
        Select-Object Id, ProcessName
}
```

Es normal que el proceso se llame `dotnet` en una publicación dependiente del framework. No debes interpretar ese nombre como si el navegador ejecutara C#; el navegador sigue enviando HTTP y el proceso se encarga de servir las solicitudes.

Ahora ejecuta en una tercera terminal la misma publicación con el mismo puerto y con el mismo modo sin base:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location .\.work\M05\publicacion-mvc
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

El segundo arranque debe fallar al intentar escuchar la dirección ocupada, siempre que la primera instancia siga viva y no haya otra particularidad de configuración. Comprueba que el primer proceso continúa respondiendo a `/Productos/Estado`; el fallo del segundo no ha detenido automáticamente al primero.

Restauración: cierra la terminal del segundo proceso con Ctrl+C si continúa abierta y ejecuta `Pop-Location`. No mates por identificador un proceso que no te pertenece. La prueba consiste en comprender la colisión, no en practicar eliminación indiscriminada de procesos.

Paso P5.3.8. Cambiar únicamente el puerto y comprobar la independencia del código

Relación con la teoría: 5.3.9 y 5.3.10. Mantén el primer servidor activo en 5091. Desde la segunda terminal, inicia otra instancia de la misma DLL, esta vez en 5094:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location .\.work\M05\publicacion-mvc
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5094
```

La segunda instancia debe poder arrancar si 5094 está libre. No hemos modificado `HomeController`, `ProductosController`, ninguna vista ni el archivo `.csproj`. Solo hemos indicado una dirección de escucha diferente.

Desde otra terminal compara:

```
curl.exe -i http://localhost:5091/Productos/Estado
curl.exe -i http://localhost:5094/Productos/Estado
```

Ambas respuestas deben proceder de la misma versión de MVC A, aunque atendidas por procesos distintos. Detén primero la instancia 5094 y después la 5091, comprobando que al cerrar una no desaparece necesariamente la otra. Vuelve a la raíz con `Pop-Location` en cada terminal.

Pregunta razonada: ¿por qué no habría solucionado la colisión cambiar la URL de una vista Razor? Porque la colisión se produce al abrir el socket de escucha durante el arranque, antes de procesar el HTML de esa vista.

Paso P5.3.9. Comprobar un fallo de arranque sin diagnosticarlo como fallo HTTP

Un caso diferente aparece si la DLL publicada no puede iniciar por una dependencia previa. En la configuración normal de MVC A, el host prepara LocalDB durante el arranque. Si la instancia falta o no se puede abrir, puede fallar antes de empezar a escuchar.

Para no provocar daños en la base ni en la configuración existente, no elimines instancias ni archivos SQL. En su lugar, razona sobre la salida de error y comprueba primero el estado de la variable `Despliegue__SinLocalDb`. Si estaba activada, el comportamiento 503 de Productos es intencionado. Si estaba desactivada, el host intentará preparar la conexión LocalDB real. En cualquier caso, antes de atribuir un error a Kestrel comprueba si llegó a aparecer una dirección de escucha y si alguna ruta devolvió HTTP.

Este paso no exige forzar una avería en la máquina del docente. Consiste en reconocer la evidencia que distinguiría una excepción de arranque, un puerto ocupado y una acción que responde 503. Un alumno capaz de explicar esas tres posibilidades puede trasladar el diagnóstico al siguiente hospedaje sin confundir subsistemas.

Paso P5.3.10. Cierre reproducible de la variante

Después de detener todas las instancias Kestrel abiertas para el ensayo, en cada terminal que hubiera configurado la opción temporal ejecuta:

```
Remove-Item Env:\Despliegue__SinLocalDb -ErrorAction SilentlyContinue
```

Comprueba que la carpeta `.work\M05\publicacion-mvc` continúa disponible. No elimines el proyecto original ni su `ConnectionStrings:Productos`. El siguiente punto seguirá utilizando la **misma publicación MVC**, pero con un sitio IIS que determinará su propio mecanismo de entrada.

El resultado que debes ser capaz de explicar es una secuencia completa: arranque de un proceso desde una DLL publicada, lectura de su puerto, solicitud de varias rutas, diagnóstico de un segundo

arranque conflictivo, ejecución en un puerto alternativo y parada controlada. Todos esos pasos pertenecen a un único proyecto y pueden reproducirse sin alterar el proveedor de datos.

Paso P5.3.11. Preparar un recorrido de solicitudes con evidencia suficiente

Fundamento: 5.3.23–5.3.25. El objetivo de la actividad no es limitarse a abrir la portada. Queremos demostrar que la misma publicación puede servir una vista MVC, una acción de controlador y un recurso estático.

Desde la raíz del repositorio identifica la publicación preparada en P5.2. Si no existe, vuelve a ejecutar `dotnet publish` con el proyecto MVC original y una salida conocida bajo `.work\M05`. No crees una nueva solución MVC.

En una terminal PowerShell configura explícitamente el escenario sin LocalDB para aislar el hospedaje:

```
$salida = ".\.work\M05\publicacion-mvc"
$env:Despliegue__SinLocalDb = "true"
Push-Location $salida
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

Mantén abierta la terminal. Observa la dirección de escucha que anuncia el host. Si la configuración efectiva utiliza otra dirección, no continúes solicitando 5091 sin comprobar la causa.

En otra terminal, desde cualquier carpeta, consulta la página inicial:

```
curl.exe -i http://localhost:5091/
```

Localiza una respuesta HTML correspondiente al recorrido inicial de MVC A. Una página con estado 200 es la primera evidencia, no el final de la prueba.

Solicita la acción de estado:

```
curl.exe -i http://localhost:5091/Productos/Estado
```

Debe devolver el texto `Catalogo MVC disponible`. La acción no consulta SQL Server; por eso constituye un buen punto de observación del host y del routing.

Solicita por último el CSS:

```
curl.exe -i http://localhost:5091/css/site.css
```

Esta respuesta permite comprobar que se ha publicado un recurso bajo `wwwroot` y que el middleware lo sirve. Si falla mientras Estado funciona, la causa no es necesariamente el puerto: el proceso sí está atendiendo otras rutas.

Paso P5.3.12. Distinguir una ruta inexistente de un servidor inexistente

Fundamento: 5.3.30 y 5.3.33. Con Kestrel todavía iniciado solicita una ruta que no figura en MVC A:

```
curl.exe -i http://localhost:5091/ruta-que-no-existe
```

Es razonable esperar un 404. Observa el código, no solo el aspecto del navegador. La respuesta demuestra que un proceso recibió la solicitud, pero no existe un endpoint que produzca el recurso esperado.

Ahora detén Kestrel con Ctrl+C. Después, desde otra terminal, repite la consulta de Estado:

```
curl.exe -i http://localhost:5091/Productos/Estado
```

Si no hay ningún otro proceso utilizando 5091, obtendrás un error de conexión en lugar de una respuesta 404 de MVC. No atribuyas automáticamente un código HTTP a un fallo en el que no se recibió respuesta alguna.

Pregunta razonada: ¿por qué el 404 demuestra más acerca de la conectividad que una ventana del navegador que simplemente informa de que no puede establecer conexión? Porque el 404 es una respuesta generada después de que el servidor atienda una solicitud, mientras que el segundo caso puede no haber llegado a ningún endpoint.

Vuelve a iniciar Kestrel antes de seguir y comprueba que Estado responde otra vez. La capacidad de recuperarse sin reconstruir el proyecto es parte del aprendizaje.

Paso P5.3.13. Investigar qué proceso escucha antes de detenerlo

Fundamento: 5.3.28. Con el servidor en marcha en 5091 abre una tercera terminal PowerShell y ejecuta:

```
$escuchas = Get-NetTCPConnection -State Listen -LocalPort 5091 -ErrorAction SilentlyContinue
$escuchas | Select-Object LocalAddress, LocalPort, OwningProcess
```

Cuando exista una fila, consulta su proceso:

```
$escuchas | ForEach-Object {
    Get-Process -Id $_.OwningProcess -ErrorAction SilentlyContinue |
        Select-Object Id, ProcessName, Path
}
```

Compara el resultado con la terminal donde ejecutaste `dotnet CursoWeb.Mvc.dll`. No elimines procesos automáticamente por identificador. Si el puerto pertenece a otra aplicación y no la controlas, elige otro puerto para el ensayo y comunica el cambio al docente.

Esta actividad corrige una práctica peligrosa: detener cualquier proceso que aparezca usando un puerto. El dato `OwningProcess` es una pista para investigar, no una autorización para cerrar aplicaciones ajenas.

Paso P5.3.14. Distinguir la ejecución de proyecto de la ejecución de publicación

Fundamento: 5.3.31. Tenemos dos comandos que pueden producir la misma interfaz:

```
dotnet run --project .
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj --no-launch-
profile --urls http://localhost:5091
```

```
Push-Location .\.work\M05\publicacion-mvc
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

No ejecutes ambos a la vez en el mismo puerto. El primer comando utiliza el contexto de proyecto y puede compilar; el segundo ejecuta los artefactos publicados. Si la versión de archivos es la misma, la página recibida puede verse idéntica, pero la fuente del proceso es distinta.

Realiza una prueba secuencial: ejecuta la primera modalidad, abre Estado, detén el proceso; ejecuta la segunda, abre Estado, detén el proceso. Anota qué cambió y qué se mantuvo. Los controladores, vistas y rutas pertenecen a la misma MVC A. El comando y la carpeta desde la que se ejecutan sí han cambiado.

Corrección de una interpretación frecuente: Kestrel no aparece solo en la ejecución desde Visual Studio. También puede atender solicitudes al ejecutar la publicación. El mecanismo HTTP es el mismo, aunque el camino para crear e iniciar los artefactos sea distinto.

Paso P5.3.15. Provocar una colisión de puerto y verificar que la primera instancia sigue viva

Fundamento: 5.3.26–5.3.27. Utiliza el modo sin base para que el experimento no dependa de una instancia SQL Server. Abre una terminal y ejecuta la DLL publicada en 5091. Comprueba Estado por HTTP.

Sin cerrar la primera, abre otra terminal con la misma carpeta de publicación y el mismo puerto. El segundo proceso debería informar de una dirección ocupada, siempre que llegue a la fase de escucha. No cambies la ruta Home/Index ni el programa fuente para resolverlo.

Después del fallo del segundo proceso, solicita otra vez:

```
curl.exe -i http://localhost:5091/Productos/Estado
```

Debe continuar respondiendo el primero. Así demuestras que la colisión del segundo arranque no equivale a la caída de la instancia inicial.

Detén el segundo proceso si continuara ejecutándose y libera el puerto principal solo al acabar el ensayo. No dejes dos terminales que parezcan estar sirviendo la misma aplicación.

Paso P5.3.16. Repetir el ensayo en un puerto alternativo sin cambiar una vista Razor

Fundamento: 5.3.29. Mantén la instancia original en 5091. En otra terminal inicia la misma DLL en 5094:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location .\.work\M05\publicacion-mvc
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5094
```

En una tercera terminal compara:

```
curl.exe -i http://localhost:5091/Productos/Estado
curl.exe -i http://localhost:5094/Productos/Estado
```

Ambas deberían devolver el mismo texto. No hemos creado otro proyecto, sino otra instancia de la misma publicación. La dirección de escucha pertenece al proceso y no exige duplicar las acciones MVC.

Detén la instancia 5094 y comprueba que 5091 sigue respondiendo. Ahora detén también 5091. El resultado final es que ambos puertos quedan libres para otras prácticas. Ejecuta `Pop-Location` en las terminales en que lo utilizaste y elimina la variable temporal de cada una.

Paso P5.3.17. Describir por qué una portada correcta no acredita un catálogo persistente

Fundamento: 5.3.32. Vuelve a iniciar la publicación en modo sin LocalDB, esta vez únicamente durante la comprobación:

```
$env:Despliegue__SinLocalDb = "true"
Push-Location .\.work\M05\publicacion-mvc
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

Abre `/`, `/Productos/Estado` y `/Productos` en pestañas distintas. Las dos primeras rutas deben funcionar y la última debe devolver 503. Es el resultado definido por el proyecto: el middleware impide mostrar un CRUD inexistente en ese modo de ejecución.

Describe por qué no es correcto afirmar «la aplicación no funciona» sin precisar qué parte falla. También sería incorrecto afirmar «el CRUD funciona» por la portada. Lo que sabemos es que el hospedaje permite servir determinadas rutas mientras una funcionalidad dependiente de SQL Server está deshabilitada.

No introduzcas SQLite, no modifiques `UseSqlServer` y no elimines el middleware de protección. Eso alteraría la tecnología acordada y eliminaría el límite que este ejercicio se propone reconocer.

Paso P5.3.18. Resolver tres incidentes a partir de observaciones diferentes

Incidente A: la ejecución falla antes de mostrar una dirección de escucha. Se informa de un error de conexión a LocalDB. Antes de intervenir sobre Kestrel, comprueba si se utilizó el modo normal y si la base existe bajo el usuario actual. Para verificar solo el hospedaje puede activarse explícitamente el modo sin base, sin fingir que el CRUD se ha probado.

Incidente B: la aplicación anuncia el puerto 5091 y la portada funciona, pero `/ruta-que-no-existe` devuelve 404. No reconstruyas el proyecto. El servidor ha respondido correctamente que esa ruta no está definida. Para comprobar MVC utiliza una ruta conocida, como Estado.

Incidente C: Estado devuelve el texto esperado en 5094, pero el alumno sigue abriendo 5091. No cambies el controlador para que «responda» al puerto antiguo. Comprueba cuál fue la dirección indicada al iniciar el proceso y actualiza la URL solicitada.

Cada incidente exige identificar un punto diferente del recorrido: dependencia previa, selección de ruta y dirección efectiva de escucha. La explicación forma parte del resultado, no es una pregunta ornamental.

Paso P5.3.19. Repetición final sin ayudas de copia

Cierra las terminales de procesos previos. Desde la raíz del repositorio, sin modificar código, prepara la publicación si no existe; inicia la DLL en un puerto local libre y anota ese puerto.

Después solicita una vista, una acción de texto y una hoja CSS. Observa también la respuesta de Productos bajo la configuración elegida y explica por qué puede ser 503.

Detén el proceso y comprueba que no has dejado un listener de tu aplicación ocupando el puerto. Esta repetición final reconstruye el ciclo de vida completo: artefactos, proceso, dirección, solicitud, respuesta e interrupción controlada.

Estado de salida: una publicación MVC A que ha respondido por Kestrel y cuyo resultado se puede vincular a un proceso y a una configuración determinados. IIS reutilizará sus artefactos y rutas, pero añadirá sitio, pool y módulo ASP.NET Core.

Diagnóstico P5.3: qué comprobar y por qué

Problema	Comprobación directa	Corrección proporcionada
DLL no encontrada	`Get-Location`, `Get-ChildItem`	Entrar en la publicación real
Runtime ausente	`dotnet --list-runtimes`	Instalar ASP.NET Core Runtime 8 compatible
Puerto ocupado	Listener en el puerto	Detener proceso propio o elegir otro
Fallo SQL al arrancar	Mensaje de LocalDB	Revisar instancia o habilitar modo sin base para hospedaje
CSS responde 404	Archivos en `wwwroot` y ruta	Revisar publicación y middleware estático
`/Productos` devuelve 503 en modo sin base	Valor de `Despliegue__SinLocalDb`	Interpretar la limitación; desactivarla solo con LocalDB preparado

Resultado del punto

Ya hemos visto la misma aplicación MVC ejecutándose a partir del proyecto y a partir de su publicación. Entendemos que Kestrel recibe peticiones en una dirección determinada y sabemos separar un problema de puerto, una dependencia de ejecución y una ruta que usa SQL Server. El siguiente punto cambiará el mecanismo de hospedaje, no el código de nuestros controladores.

P5.4. Publicar la aplicación MVC en IIS

Objetivo y prerrequisitos

Seguimos trabajando con la **misma aplicación MVC A** que acabamos de publicar y ejecutar con Kestrel. Ahora la alojaremos mediante Internet Information Services en Windows. Esta actividad no consiste en convertir MVC en páginas HTML estáticas ni en instalar SQL Server dentro de IIS.

Antes de comenzar necesitamos:

- Windows con la característica IIS habilitada y permisos para administrar un sitio local de laboratorio.
- ASP.NET Core Hosting Bundle 8 instalado, con el módulo ASP.NET Core registrado en IIS.
- La carpeta de publicación creada en P5.2 o la posibilidad de volver a ejecutar `dotnet publish`.
- Una carpeta de destino que no sea el proyecto fuente y un puerto local libre, por ejemplo 8091.
- Una cuenta del laboratorio autorizada para crear un sitio y configurar el Application Pool.

Importante: una instalación de Windows que no incluya IIS o en la que no se disponga del Hosting Bundle no permite declarar esta práctica verificada. En ese caso conserva las instrucciones y continúa solo con aquello que pueda comprobarse sin falsear resultados.

Paso P5.4.1. Identificar las piezas antes de configurarlas

En el Administrador de IIS distinguiremos estos elementos:

Elemento	Decisión de laboratorio
Sitio web	`CursoWeb-M05`
Ruta física	`C:\CursoWeb\M05\IIS`
Binding	`http`, puerto `8091`, host local
Application Pool	`CursoWeb-M05-Pool`
Versión CLR del pool	`No Managed Code`
Aplicación	Salida publicada de `CursoWeb.Mvc`
Configuración de base	Modo de hospedaje sin LocalDB para esta demostración

La expresión `No Managed Code` no significa que ASP.NET Core deje de ejecutar código administrado. Solo indica que el pool IIS no debe seleccionar un CLR clásico de .NET Framework para hospedar una aplicación moderna .NET 8. El Hosting Bundle y ANCM proporcionan el mecanismo correspondiente.

El sitio y el pool son **recursos de laboratorio**. No debes cambiar el Default Web Site de otras aplicaciones ni utilizar carpetas de producción como destino del experimento.

Paso P5.4.2. Preparar el directorio de publicación para IIS

Desde PowerShell, en la raíz del repositorio, prepara dos rutas:

```
$proyecto = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj"
$destinoIis = "C:\CursoWeb\M05\IIS"
New-Item -ItemType Directory -Path $destinoIis -Force | Out-Null
dotnet publish $proyecto -c Release -o $destinoIis
```

No copies la carpeta del proyecto sin publicar. Comprueba que `CursoWeb.Mvc.dll`, `CursoWeb.Mvc.runtimeconfig.json`, `wwwroot` y `web.config` se encuentran en la ruta de destino. Si falta el `web.config`, revisa que publicaste el proyecto web correcto y que el SDK generó soporte para IIS.

Abre el `web.config` publicado para identificar el elemento `aspNetCore`, que informa al módulo de cómo iniciar la aplicación. **No reemplaces el archivo por una plantilla genérica sin entender sus valores.** La publicación suele generar esos valores a partir del proyecto.

Paso P5.4.3. Aislar explícitamente la dependencia de LocalDB

Durante M02 ejecutamos el CRUD MVC desde Visual Studio con una instancia LocalDB asociada al usuario del alumno. IIS ejecutará la aplicación con la identidad del Application Pool, que normalmente no tiene acceso directo a esa instancia. Para la **demostración básica de**

hospedaje, crearemos una configuración de entorno específica, sin tocar el proyecto fuente ni sustituir `UseSqlServer()`.

En el directorio físico **ya publicado** crea un archivo nuevo `appsettings.Production.json` con este contenido:

```
{
  "Despliegue": {
    "SinLocalDb": true
  }
}
```

Esta clave evita inicializar la base al arrancar y permite mostrar el inicio, el estado y los recursos estáticos. Al mismo tiempo, provoca que las rutas que dependen del CRUD respondan 503 para no ocultar una dependencia que no está disponible.

El archivo presupone que el entorno del proceso IIS es `Production`, que es el valor predeterminado si no se configura otro. Si el servidor IIS del laboratorio fija explícitamente un entorno diferente, debe usarse un archivo `appsettings.<Entorno>.json` correspondiente o configurar de forma explícita la clave en ese sitio; **no afirmes que la configuración se ha aplicado sin comprobar el comportamiento**.

No modifiques `ConnectionStrings:Productos` para apuntar a una base de datos inventada. El CRUD completo continúa disponible en Windows desde Visual Studio/Kestrel con LocalDB real, fuera de este modo simplificado.

Paso P5.4.4. Crear y configurar el pool y el sitio

Abre **Administrador de Internet Information Services (IIS)** como administrador. En «Grupos de aplicaciones» crea `CursoWeb-M05-Pool` y selecciona «Sin código administrado / No Managed Code». Mantén la identidad predeterminada de ApplicationPoolIdentity salvo que el docente haya preparado una identidad específica, y no concedas permisos a Everyone como solución a un error.

En «Sitios», utiliza «Agregar sitio web» y define:

1. Nombre: `CursoWeb-M05`.
2. Ruta física: `C:\CursoWeb\M05\IIS`.
3. Tipo: `http`.
4. Dirección IP: la opción local prevista en el laboratorio.
5. Puerto: `8091`.
6. Nombre de host: vacío para un ensayo local cuando no haya conflicto.
7. Application Pool: `CursoWeb-M05-Pool`.

Comprueba que el puerto 8091 no lo utiliza ya otro sitio. Un fallo de binding es una incidencia de IIS que debe solucionarse antes de interpretar la respuesta de MVC.

Paso P5.4.5. Verificar permisos y aplicación publicada

El proceso IIS necesita permiso de lectura sobre los archivos publicados. Desde las propiedades de seguridad de la carpeta, identifica la cuenta que realmente ejecuta el pool y verifica los permisos mínimos necesarios. La asignación concreta depende del laboratorio Windows preparado y de su política de cuentas.

No otorgues `Full Control` a todos los usuarios para sortear un error de acceso. Si el sitio no inicia, consulta el diagnóstico de IIS y del módulo ASP.NET Core antes de cambiar permisos. Si instalaste el Hosting Bundle recientemente, puede ser necesario reiniciar los servicios IIS para cargar el módulo según su documentación.

Paso P5.4.6. Abrir las rutas que sí prueban hospedaje

En el navegador del mismo host abre:

- `http://localhost:8091/`
- `http://localhost:8091/Productos/Estado`
- `http://localhost:8091/css/site.css`

La página inicial debe proceder de la aplicación MVC A y la ruta de estado debe responder `Catalogo MVC disponible`. Comprueba también que la hoja CSS se sirve desde `wwwroot`. La prueba de estas rutas muestra que IIS, ANCM, la aplicación publicada y el routing cooperan correctamente.

Prueba a continuación `http://localhost:8091/Productos`. Bajo el modo `SinLocalDb=true`, la respuesta prevista es **503 Service Unavailable** con una explicación de que el CRUD del ensayo requiere LocalDB. Esta respuesta es deliberada: no la anotes como prueba de consulta de productos satisfactoria.

Si la ruta de Productos ejecuta SQL y falla de otra manera, comprueba que el archivo `appsettings.Production.json` está en la carpeta física correcta y que el entorno efectivo es Production. No presupongas que IIS lee la configuración que modificaste: compruébalo por la respuesta del programa.

Paso P5.4.7. Diagnosticar un fallo sin borrar y rehacer todo

Observación	Diagnóstico inicial	Corrección orientada
Sitio no arranca	Hosting Bundle, ANCM, web.config, logs	Confirmar instalación y ruta publicada
Puerto 8091 ocupado	Binding IIS o proceso externo	Elegir binding libre o detener solo proceso propio
Error de lectura de archivos	Identidad del pool	Permisos mínimos sobre carpeta
Home responde, CSS no	wwwroot y publicación	Revisar archivos publicados
Home responde, Productos devuelve 503	Modo sin LocalDB activo	Resultado previsto del ensayo
Productos intenta abrir LocalDB	Configuración de entorno no aplicada	Revisar Production y archivo en destino
Aparece una web antigua	Ruta física o sitio equivocados	Comprobar carpeta, sitio y binding

En una instalación que no proporcione IIS real, esta tabla ayuda a estudiar causas, pero **no sustituye la ejecución del sitio**. Tampoco un `dotnet run` satisfactorio justifica escribir «IIS validado»: son mecanismos de hospedaje distintos.

Paso P5.4.8. Conservar o desmontar el laboratorio

Al terminar registra el nombre del sitio, el binding utilizado, la carpeta publicada y las rutas realmente comprobadas. Si el equipo es compartido, el docente indicará si se debe detener el sitio y eliminar la configuración de laboratorio. No borres otros sitios ni el CRUD original de M02.

Estado final: mismo MVC A, publicado a una carpeta gestionada por IIS, con respuesta comprobada en rutas sin base y con una explicación explícita de por qué el ensayo no acredita acceso a LocalDB bajo la identidad del pool.

Paso P5.4.9. Relacionar la carpeta publicada con el sitio que realmente la utiliza

Relación con la teoría: 5.4.8 y 5.4.9. Este recorrido de observación complementa la creación del sitio que hicimos con la interfaz gráfica. No es necesario instalar otra característica ni crear un segundo sitio: debemos identificar los recursos del mismo laboratorio.

En una ventana PowerShell con permisos adecuados sobre IIS, importa el módulo administrativo:

```
Import-Module WebAdministration
Get-Website -Name "CursoWeb-M05" |
    Select-Object Name, State, PhysicalPath, ApplicationPool

Get-WebBinding -Name "CursoWeb-M05" |
    Select-Object Protocol, BindingInformation
```

Compara la ruta física mostrada con `C:\CursoWeb\M05\IIS`, que elegiste al publicar. Después confirma que esa carpeta contiene los artefactos de aplicación:

```
$destinoIis = "C:\CursoWeb\M05\IIS"

Test-Path (Join-Path $destinoIis "CursoWeb.Mvc.dll")
Test-Path (Join-Path $destinoIis "CursoWeb.Mvc.runtimeconfig.json")
Test-Path (Join-Path $destinoIis "web.config")
Test-Path (Join-Path $destinoIis "wwwroot\css\site.css")
```

Los cuatro resultados deberían ser `True` en una publicación correcta del proyecto. Si IIS apunta a otra ruta, no tendrá por qué servir el contenido que acabas de publicar, aunque `dotnet publish` haya terminado sin errores.

Interpretación: la publicación prepara archivos y el sitio IIS decide qué carpeta física utiliza. Se trata de dos operaciones diferentes. Una página que sigue mostrando contenido antiguo puede deberse a que IIS esté leyendo otra carpeta, no necesariamente a un fallo de compilación.

Paso P5.4.10. Verificar el Application Pool sin confundir el CLR clásico con .NET 8

Relación con la teoría: 5.4.9, 5.4.10 y 5.4.11. La aplicación debe ejecutarse mediante ASP.NET Core Module. El Application Pool conserva una configuración particular que hemos elegido para ese mecanismo de hospedaje.

Desde PowerShell, con el módulo de IIS importado, consulta:

```
Get-ItemProperty "IIS:\AppPools\CursoWeb-M05-Pool" |
    Select-Object Name, ManagedRuntimeVersion, ProcessModel
```

```
Get-WebAppPoolState -Name "CursoWeb-M05-Pool"
```

El valor de `ManagedRuntimeVersion` configurado para «No Managed Code» suele representarse como cadena vacía. Eso no significa que nuestro proyecto haya pasado a ser una página estática sin .NET. Significa que el CLR tradicional de .NET Framework no se usa para hospedar ASP.NET Core como si fuera una aplicación antigua.

Si el pool está detenido, consulta la causa antes de iniciarlo. Si está iniciado pero la página devuelve un error, no atribuyas automáticamente el fallo al código MVC. Revisa el Hosting Bundle, el ASP.NET Core Module, los permisos y los artefactos publicados.

Pregunta guiada: ¿por qué no demuestra una propiedad `State = Started` que Home/Index devuelve HTML? Porque el estado administrativo del pool y la respuesta de una acción MVC son observaciones distintas. Solo una petición HTTP a la URL correcta permite comprobar la segunda.

Paso P5.4.11. Separar respuesta de hospedaje de respuesta de una función dependiente de SQL

Relación con la teoría: 5.4.13–5.4.15. Con el sitio operativo, realiza tres solicitudes que no necesitan acceso a LocalDB:

```
curl.exe -i http://localhost:8091/
curl.exe -i http://localhost:8091/Productos/Estado
curl.exe -i http://localhost:8091/css/site.css
```

Para una instalación preparada, deben responder 200. Ahora solicita una función que en MVC A utiliza el `AppDbContext` de SQL Server:

```
curl.exe -i http://localhost:8091/Productos
```

Con el archivo `appsettings.Production.json` de P5.4.3 y el entorno correspondiente, la respuesta prevista es 503. No se ha fingido un catálogo en memoria ni se ha cambiado `UseSqlServer`. En este ensayo nos interesa demostrar que IIS hospeda la aplicación, no que dispone de una instancia SQL Server Express LocalDB asociada a su identidad de proceso.

Si aparece un 500 durante el arranque, no es equivalente a este 503 deliberado. Revisa qué configuración se cargó realmente y los mensajes de ANCM. Si todas las rutas fallan, el problema se sitúa probablemente antes de la acción concreta del catálogo. Si solo una ruta de datos falla y la portada funciona, el análisis deberá considerar la dependencia de base.

Por qué importa la identidad: un Application Pool utiliza una cuenta distinta de la sesión interactiva del alumno. LocalDB en Windows está asociado a un contexto de usuario. La misma cadena de conexión que funcionaba desde Visual Studio no demuestra que IIS tenga acceso autorizado a esa instancia.

Paso P5.4.12. Comparar dos publicaciones sin cambiar un solo controlador

Relación con la teoría: 5.4.9 y 5.4.14. En P5.2 aprendimos que modificar una vista de origen no sustituye automáticamente la carpeta publicada. IIS añade otra relación: el sitio sirve la carpeta física que se le configuró.

Imagina dos carpetas con publicaciones del mismo proyecto: `C:\CursoWeb\M05\IIS` y una copia de ensayo bajo `.work`. Aunque ambas contengan una DLL llamada `CursoWeb.Mvc.dll`, solo una de ellas corresponde a la ruta física del sitio `CursoWeb-M05`.

No necesitas cambiar el binding ni crear otro sitio. Consulta el valor `PhysicalPath` y comprueba que coincide con el destino al que ejecutaste `dotnet publish`. Si acabas de modificar una vista del origen y la portada IIS sigue siendo la antigua, identifica primero si repúblicaste y si el sitio utiliza la salida nueva.

El resultado esperado de este razonamiento es una explicación completa: fuente, publicación, ruta física IIS, módulo ASP.NET Core y petición HTTP. Se trata de cinco elementos distintos; cambiar uno no implica que los otros hayan cambiado.

Paso P5.4.13. Dejar el host IIS preparado para el siguiente ejercicio

La práctica Docker no necesita que IIS permanezca activo en 8091. Si el laboratorio utiliza una máquina compartida, conserva el nombre del sitio y del pool, informa al docente y detén únicamente lo que hayas creado para la sesión siguiendo las reglas del aula. No borres el Default Web Site ni otras publicaciones.

Puedes consultar el estado final sin modificarlo:

```
Get-Website -Name "CursoWeb-M05" |
    Select-Object Name, State, PhysicalPath

Get-WebAppPoolState -Name "CursoWeb-M05-Pool"
```

El conocimiento que debes conservar no es la posición de un botón de la consola administrativa, sino la relación entre el sitio, su binding, el pool, el módulo de hospedaje, los archivos publicados y el resultado de una petición concreta. Esa misma disciplina permitirá interpretar después las imágenes, los puertos y los procesos de Docker.

Paso P5.4.14. Identificar un sitio IIS con una consulta administrativa

Fundamento: 5.4.16–5.4.18. El alumno ya ha creado el sitio mediante la interfaz gráfica. Ahora aprenderá a identificar los mismos objetos mediante PowerShell para comprobar que las opciones escogidas se han aplicado.

Abre PowerShell con las autorizaciones que requiera el laboratorio IIS e importa el módulo:

```
Import-Module WebAdministration
```

Consulta el sitio concreto sin modificarlo:

```
Get-Website -Name "CursoWeb-M05" |
    Select-Object Name, State, PhysicalPath, ApplicationPool
```

El resultado debe identificar el sitio, su estado, la ruta física publicada y el pool asignado. Una configuración correcta debería apuntar a `C:\CursoWeb\M05\IIS`, siempre que hayas utilizado la carpeta recomendada.

Consulta después su binding:

```
Get-WebBinding -Name "CursoWeb-M05" |
    Select-Object Protocol, BindingInformation
```

No basta con reconocer la cadena `8091` en la salida. Relaciona protocolo, puerto y sitio. Si la URL se abre en otro puerto, puede estar utilizando un sitio diferente y la prueba ya no demuestra el mecanismo que pretendíamos estudiar.

Resultado observable: el sitio existe y su configuración es coherente con los valores definidos en la práctica. La existencia del sitio todavía no prueba una respuesta HTTP de nuestra aplicación; esa es una comprobación independiente.

Paso P5.4.15. Preparar una variante ejecutable sin sustituir la enseñanza manual

Fundamento: 5.4.19–5.4.22. La práctica principal utiliza el Administrador de IIS para que el alumno comprenda cada elemento. Después de realizarla y comprenderla, se puede estudiar una variante PowerShell completa incluida fuera del proyecto compilable.

La variante se conserva en esta ruta:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/VARIANTES/P5.4_IIS/CrearSitioIis.ps1`

El código completo de la variante, que puedes editar antes de ejecutarlo con privilegios administrativos, es:

```
# P5.4 — Variante autónoma. Ejecutar como administrador SOLO en host Windows
# con IIS y ASP.NET Core Hosting Bundle 8 previamente instalados.
# No modifica proyectos ni elimina sitios existentes.
param(
    [string]$Proyecto = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj",
    [string]$Destino = "C:\CursoWeb\M05\IIS",
    [int]$Puerto = 8091,
    [string]$Sitio = "CursoWeb-M05",
    [string]$Pool = "CursoWeb-M05-Pool"
)
$ErrorActionPreference = "Stop"
if (-not (Test-Path $Proyecto)) { throw "Falta el proyecto MVC: $Proyecto" }
Import-Module WebAdministration -ErrorAction Stop
if (Get-Website -Name $Sitio -ErrorAction SilentlyContinue) {
    throw "El sitio $Sitio ya existe; no se sobrescribe"
}
if (Test-Path "IIS:\AppPools\$Pool") {
    throw "El pool $Pool ya existe; no se sobrescribe"
}
$bindings = @(Get-WebBinding -Protocol http | Where-Object {
    $_.bindingInformation -like "*:$($Puerto):*"
})
if ($bindings.Count -gt 0) {
    throw "El puerto HTTP $Puerto ya está reservado por otro binding IIS"
}
New-Item -ItemType Directory -Path $Destino -Force | Out-Null
dotnet publish $Proyecto -c Release -o $Destino
if ($LASTEXITCODE -ne 0) { throw "No se pudo publicar MVC" }
foreach ($nombre in @("CursoWeb.Mvc.dll", "web.config", "wwwroot\css\site.css")) {
    if (-not (Test-Path (Join-Path $Destino $nombre))) {
        throw "La publicación no contiene $nombre"
    }
}
# Se modifica solo el directorio publicado, no el appsettings del proyecto.
```

```
$ajustes = @{ Despliegue = @{ SinLocalDb = $true } }
$ajustes | ConvertTo-Json -Depth 4 |
    Set-Content -Path (Join-Path $Destino "appsettings.Production.json") -Encoding UTF8
New-WebAppPool -Name $Pool | Out-Null
Set-ItemProperty "IIS:\AppPools\$Pool" -Name managedRuntimeVersion -Value ""
$identidad = "IIS AppPool\$Pool"
& icacls.exe $Destino /grant "${identidad}:(OI)(CI)RX" /T | Out-Null
if ($LASTEXITCODE -ne 0) { throw "No se pudieron asignar permisos de lectura" }
New-Website -Name $Sitio -PhysicalPath $Destino -Port $Puerto -ApplicationPool $Pool | Out-Null
Start-Website -Name $Sitio
Write-Host "Sitio $Sitio preparado: http://localhost:$Puerto/"
Write-Host "Comprobar GET /, /Productos/Estado, /css/site.css y el 503 de /Productos."
Write-Host "No se ha comprobado CRUD con LocalDB bajo la identidad IIS."
```

En este código cada comprobación protege una condición del entorno: no usar nombres ocupados, no publicar fuera de la carpeta prevista y no prometer una base que IIS no puede abrir. La variante está disponible como archivo ejecutable completo en la ruta anterior; no es una transcripción parcial.

No es un nuevo proyecto ASP.NET Core. Es un archivo de automatización limitado al **sitio docente del laboratorio**. Su finalidad es enseñar qué decisiones de interfaz gráfica pueden expresarse mediante comandos y cómo evitar sobrescribir una instalación ya existente.

Preparación obligatoria: antes de usarla, confirma que Windows dispone de IIS y ASP.NET Core Hosting Bundle 8. Comprueba que el nombre de sitio y el pool elegidos no existen. Si ya completaste el recorrido principal y conservaste `CursoWeb-M05`, no ejecutes la variante con los nombres predeterminados: se ha diseñado para detectar y rechazar ese estado, no para reemplazar un sitio anterior.

Puedes conservar el sitio previo y leer la variante sin ejecutarla, o utilizar otro nombre, pool y puerto autorizado exclusivamente para un ensayo adicional. Nunca cambies ni borres el Default Web Site para liberar un nombre de práctica.

Paso P5.4.16. Entender los parámetros de un ensayo reversible

El archivo empieza con un conjunto de parámetros:

```
param(
    [string]$Proyecto = ".
\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj",
    [string]$Destino = "C:\CursoWeb\M05\IIS",
    [int]$Puerto = 8091,
    [string]$Sitio = "CursoWeb-M05",
    [string]$Pool = "CursoWeb-M05-Pool"
)
```

El alumno debe explicar cada entrada: dónde está el proyecto fuente, dónde estará la publicación, qué puerto utilizará IIS, cómo se identificará el sitio y qué pool administrará su proceso.

La variante comprueba la existencia del proyecto antes de publicar. También comprueba si existen un sitio o pool con los nombres elegidos y si hay otro binding utilizando el puerto solicitado. En vez de eliminar recursos ajenos, produce un error descriptivo. Esa precaución es esencial para que una ampliación docente no destruya configuraciones de un equipo compartido.

Para ensayar sin colisionar con el sitio principal, una ejecución posible en un host preparado sería:

```
.\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.4_IIS\CrearSitioIis.ps1 `
  -Proyecto ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc\CursoWeb.Mvc.csproj" `
  -Destino "C:\CursoWeb\M05\IIS-Ensayo" `
  -Puerto 8094 `
  -Sitio "CursoWeb-M05-Ensayo" `
  -Pool "CursoWeb-M05-Ensayo-Pool"
```

Este ejemplo exige PowerShell con autorización administrativa y las características IIS ya preparadas. Se ejecuta desde la raíz del repositorio para que la ruta relativa del proyecto sea correcta. No se incluye una política para borrar automáticamente el sitio: el docente supervisa cualquier retirada de recursos de IIS después de comprobar sus nombres.

Paso P5.4.17. Reconocer las operaciones que lleva a cabo la variante

En el archivo encontrarás una secuencia de acciones que corresponden a lo aprendido:

- Importar `WebAdministration` para disponer de las órdenes administrativas IIS.
- Rechazar un nombre de sitio o pool que ya exista, sin sobrescribirlo.
- Comprobar que el puerto solicitado no tenga un binding HTTP incompatible.
- Ejecutar `dotnet publish` para preparar la aplicación MVC original.
- Comprobar que existen la DLL, `web.config` y el CSS publicado.
- Escribir **solo en la carpeta publicada** un `appsettings.Production.json` que activa el modo sin LocalDB.
- Crear un pool con `No Managed Code` y conceder permisos de lectura pertinentes a su identidad.
- Crear el sitio, asociarlo al pool y comunicar su dirección local.

Lee los comandos del archivo y señala la línea responsable de cada operación. No basta con responder que «el script despliega IIS». La competencia buscada es relacionar cada instrucción con un objeto administrativo y explicar qué pasaría si fallara.

Atención a LocalDB: el script no da acceso a la base del usuario a la identidad IIS, ni pretende hacerlo. Mantiene la aplicación MVC y su proveedor SQL Server, pero activa el ensayo sin base para las rutas que no requieren persistencia.

Paso P5.4.18. Verificar la variante mediante solicitudes HTTP reales

Si has ejecutado la variante con puerto 8094 y nombres alternativos, no uses por costumbre la URL 8091 del sitio principal. Abre las siguientes direcciones:

```
curl.exe -i http://localhost:8094/
curl.exe -i http://localhost:8094/Productos/Estado
curl.exe -i http://localhost:8094/css/site.css
curl.exe -i http://localhost:8094/Productos
```

Las tres primeras deberían responder satisfactoriamente cuando IIS, ANCM y la publicación están correctos. La última debe devolver 503 por el modo sin LocalDB que la variante configura. Un resultado diferente requiere comprobar la configuración efectiva del proceso; no se resuelve cambiando el proveedor EF Core.

Después consulta la ruta física del sitio de ensayo:

```
Get-Website -Name "CursoWeb-M05-Ensayo" |
    Select-Object Name, State, PhysicalPath, ApplicationPool
```

Comprueba que corresponde al destino utilizado al ejecutar la variante. Si muestra otra carpeta, no puedes atribuir las respuestas del navegador a los archivos que pretendías publicar.

Paso P5.4.19. Explicar por qué la automatización no sustituye la práctica principal

Es posible automatizar la creación de un sitio sin comprender qué significa un binding o un Application Pool. Por eso el aprendizaje comienza con la interfaz gráfica y continúa con una variante cuyos comandos ya pueden explicarse.

El estudiante debe ser capaz de leer `New-Website` y relacionarlo con «Agregar sitio web», de reconocer la configuración `managedRuntimeVersion` del pool y de identificar por qué el módulo ASP.NET Core es necesario aunque IIS funcione con otras aplicaciones.

Un fallo de `New-Website` por nombre duplicado no exige borrar un sitio existente a ciegas. Un error de publicación no debe resolverse instalando una base nueva. Una respuesta HTTP 503 en Productos, cuando se ha activado explícitamente el modo sin base, no representa un fallo de ANCM.

La automatización solo es útil cuando preserva la comprensión del recorrido y evita introducir riesgos innecesarios en el entorno.

Paso P5.4.20. Documentar el estado final del hospedaje

Al terminar el laboratorio IIS deben poder describirse cinco hechos independientes: qué carpeta contiene la publicación, qué sitio y binding utiliza IIS, bajo qué pool se ejecuta, qué módulo y runtime soportan ASP.NET Core 8 y qué respuesta produce una ruta conocida.

No se exige instalar SQL Server bajo la identidad del pool ni convertir el CRUD de M02 en una aplicación sin persistencia. El proyecto original conserva su dependencia de LocalDB y puede reutilizarse en Windows con el contexto adecuado.

Si utilizaste la variante adicional, entrega al docente los nombres concretos del sitio y el pool para decidir una retirada controlada. No suprimas recursos cuya pertenencia no puedas identificar. Conserva la carpeta de proyecto MVC A y, si el docente lo permite, el directorio publicado de referencia para comparar con el siguiente hospedaje.

Conclusión: la publicación IIS no es una modificación del controlador MVC. Es un mecanismo de alojamiento que combina artefactos, sitio, binding, pool, módulo y permisos. La práctica queda comprendida cuando el alumno puede explicar todas esas piezas y demostrar una respuesta HTTP del sitio real.

Paso P5.4.21. Distinguir un recurso estático ausente de una aplicación detenida

Fundamento: 5.4.18 y 5.4.23. Un sitio IIS puede responder correctamente a la portada MVC y fallar al servir un CSS. Queremos observar esa diferencia controladamente, sin cambiar los archivos fuente del proyecto ni modificar permisos.

Preparación. Necesitamos el sitio `CursoWeb-M05` funcionando en `http://localhost:8091/` y su directorio físico publicado `C:\CursoWeb\M05\IIS`. Detén cualquier otro ensayo que utilice el mismo puerto. Antes de tocar archivos, confirma el estado correcto:

```
curl.exe -i http://localhost:8091/
curl.exe -i http://localhost:8091/css/site.css
```

Ambas solicitudes deben producir respuestas satisfactorias. No continúes si CSS ya falta: primero debes encontrar el motivo de esa avería inicial, porque no podrías atribuir el resultado posterior a este experimento.

Desde una PowerShell con permiso para trabajar en la carpeta publicada, define los nombres exactos:

```
$css = "C:\CursoWeb\M05\IIS\wwwroot\css\site.css"
$copia = "$css.p5421-original"
Test-Path $css
```

Si `Test-Path` devuelve `False`, la ruta física del sitio podría ser diferente; consúltala en IIS antes de continuar. No busques un CSS parecido en cualquier carpeta del equipo.

Paso P5.4.22. Retirar temporalmente solo el archivo publicado

Objetivo: demostrar que un 404 de CSS no equivale a un fallo del host ASP.NET Core.

Copia primero el archivo y después retira temporalmente la copia servida:

```
Copy-Item $css $copia -Force
Rename-Item -Path $css -NewName "site.css.p5421-oculto"
```

No borres la hoja de estilos del proyecto fuente. La modificación se realiza exclusivamente en la publicación usada por IIS y debe durar solo lo necesario para observar su efecto. Si el equipo es compartido, realiza el ensayo con autorización docente, ya que las páginas que dependan de ese CSS pueden verse temporalmente sin su formato habitual.

Solicita ahora tres URLs desde otra terminal:

```
curl.exe -i http://localhost:8091/
curl.exe -i http://localhost:8091/Productos/Estado
curl.exe -i http://localhost:8091/css/site.css
```

Si el sitio sigue funcionando, las rutas MVC pueden responder 200 mientras el CSS responde 404. La portada puede verse sin estilos al abrirla en el navegador. No es un error del controlador Home/Index: la solicitud al recurso estático no encuentra el archivo que pretendía servir.

Pregunta razonada: ¿por qué no debemos reinstalar el Hosting Bundle para resolver este escenario? Porque las rutas ASP.NET Core siguen atendiendo solicitudes; conocemos exactamente qué archivo retiramos del directorio publicado y qué recurso dejó de existir.

Paso P5.4.23. Restaurar el recurso y comprobar la recuperación

Cierra la observación recuperando la hoja original. Desde la misma terminal, con las variables definidas:

```
$oculto = Join-Path (Split-Path $css -Parent) "site.css.p5421-oculto"
if (Test-Path $oculto) {
    Remove-Item $oculto
}
Copy-Item $copia $css -Force
Remove-Item $copia
```

La restauración comprueba que el archivo original vuelve a encontrarse en la ruta publicada. No reconstruye la aplicación MVC ni modifica sus controladores. Consulta otra vez:

```
curl.exe -i http://localhost:8091/css/site.css
```

La respuesta prevista es 200. Si aún aparece 404, comprueba que el sitio apunta realmente a `C:\CursoWeb\M05\IIS` y que el archivo restaurado tiene el nombre correcto. No declares la práctica terminada hasta devolver al laboratorio su estado inicial.

Resultado de aprendizaje: una aplicación puede estar disponible mientras un recurso estático concreto no lo está. El código HTTP y el tipo de recurso nos ayudan a localizar la avería.

Paso P5.4.24. Diferenciar una copia publicada de una modificación de código fuente

Fundamento: 5.2.13–5.2.16 y 5.4.18. Acabamos de modificar temporalmente un CSS en la carpeta publicada y de observar el efecto. Ese ensayo no cambia necesariamente el archivo `wwwroot/css/site.css` del proyecto original.

Comprueba en Visual Studio que el CSS fuente conserva su contenido y que el proyecto sigue siendo MVC A de M02. Si ahora volvieras a publicar desde ese proyecto, los archivos generados podrían sustituir la copia del destino por el contenido del origen. Esa es precisamente la frontera que queremos enseñar.

El comportamiento de una vista Razor puede ser distinto: las vistas suelen compilarse como parte de la publicación, por lo que editar el archivo de origen no reemplaza automáticamente su contenido en la aplicación alojada. Por eso P5.2 construyó dos carpetas con publicaciones de distinta versión en lugar de asumir que una modificación del editor actualizaba todos los servidores.

La comparación no exige modificar ahora otra vez Home/Index ni crear un segundo sitio. Basta con señalar la diferencia causal entre un recurso estático servido desde una carpeta física y una vista dinámica que forma parte de los artefactos de la aplicación.

Paso P5.4.25. Comprobar el entorno efectivo mediante la limitación prevista

Fundamento: 5.4.20. La práctica principal creó `appsettings.Production.json` con `Despliegue:SinLocalDb=true`. No necesitamos añadir una ruta especial para revelar el entorno a través de HTTP. Podemos observar una consecuencia concreta de esa configuración.

Con el sitio IIS operativo y el CSS restaurado, consulta:

```
curl.exe -i http://localhost:8091/Productos/Estado
curl.exe -i http://localhost:8091/Productos
```

La primera ruta debe responder 200; la segunda, 503. Si Productos intenta acceder a LocalDB o falla durante el arranque, puede que la opción no se haya aplicado al proceso. Comprueba la existencia del archivo de entorno en la **ruta física publicada** y el entorno utilizado por la aplicación.

No cambies una instancia SQL ni añadas otro proveedor EF para conseguir un código diferente. El objetivo del ensayo es comprobar IIS y reconocer una limitación deliberada, no transformar el CRUD original.

Paso P5.4.26. Elegir la siguiente comprobación según el error observado

Observa estos cinco casos y explica qué investigarías primero.

Caso 1. El sitio aparece Started, pero ninguna de las rutas responde y la consola informa de un error de hospedaje. Revisa ANCM, Hosting Bundle, runtime y publicación. No se ha demostrado que el controlador se esté ejecutando.

Caso 2. El sitio y la portada responden, pero el CSS devuelve 404. Revisa `wwwroot`, la ruta de archivo y la carpeta física publicada. La prueba controlada anterior demuestra cómo puede producirse esta situación.

Caso 3. La portada devuelve 200, Estado devuelve 200 y Productos devuelve 503 bajo el modo sin base. El resultado es coherente con el alcance del ensayo. No lo etiquetes como CRUD exitoso ni como IIS caído.

Caso 4. IIS muestra una portada antigua aunque la publicación más reciente se encuentra en otra carpeta. Comprueba qué ruta física utiliza el sitio y cuál fue el destino real de `dotnet publish`.

Caso 5. La ejecución directa Kestrel responde, pero el sitio IIS no inicia. La diferencia orienta a los componentes específicos de IIS: pool, módulo, permisos y ruta física. No afirmes que la prueba Kestrel equivale a haber probado IIS.

Cada respuesta debe mencionar un archivo, una opción de configuración o una observación concreta. «Volver a empezar» no constituye un diagnóstico.

Paso P5.4.27. Recuperar el estado inicial antes de pasar a Docker

Antes de iniciar la última práctica, confirma que `wwwroot/css/site.css` está presente en la publicación IIS, que el sitio conserva sus bindings originales y que no quedan archivos temporales del experimento.

La carpeta de proyecto MVC A sigue intacta. No se ha modificado su modelo de productos, el proveedor EF Core ni la cadena de conexión. La publicación IIS continúa demostrando páginas sin base bajo la configuración de laboratorio.

El siguiente ensayo utilizará Docker Linux. No necesita que el sitio IIS siga escuchando mientras se construye la imagen, aunque pueden coexistir en puertos diferentes. El alumno debe saber qué proceso responde a cada URL antes de comparar resultados.

Síntesis: la acción de recuperar un archivo estático, comprobar la ruta física y reconocer el estado de aplicación permite cerrar una demostración de IIS sin introducir operaciones peligrosas ni alterar la arquitectura acordada.

Ampliación autónoma P5.4: explicar una diferencia de proceso

Compara la dirección 5091 utilizada en Kestrel con 8091 de IIS. Argumenta por qué ambos pueden servir la misma acción `HomeController.Index` sin modificar su código, pero dependen de procesos de arranque, identidad y permisos diferentes. Explica además por qué un fallo 503 de `/Productos` deliberadamente producido por el modo sin base no es una avería del módulo IIS.

P5.5. Crear y ejecutar un contenedor Docker Linux

Objetivo y estado inicial

En esta última práctica utilizaremos la **misma aplicación MVC A** empleada en P5.2, P5.3 y P5.4. No crearemos una nueva solución MVC ni necesitaremos que Blazor, la API de productos, Identity/JWT o SQL Server estén ejecutándose al mismo tiempo.

El estado inicial es el proyecto MVC acumulativo y el Dockerfile incluido en su carpeta. El estado final será una imagen Docker .NET 8 construida, un contenedor real que responde HTTP desde el host, la inspección de sus logs y la retirada controlada del contenedor.

Paso P5.5.1. Confirmar el motor y el tipo de contenedor

En Windows abre Docker Desktop, comprueba que el motor esté iniciado y que utiliza **contenedores Linux**. Después abre PowerShell:

```
docker version
docker info --format "{{.OSType}}"
```

La información de servidor debe ser accesible; para este ejercicio esperamos `linux`. Si la herramienta cliente existe pero no puede conectar con el daemon, no has demostrado que Docker funcione. Debes iniciar o preparar el motor antes de intentar construir imágenes.

No instalaremos SQL Server en el contenedor ni utilizaremos Docker Compose o Kubernetes. Son contenidos de otro nivel.

Paso P5.5.2. Abrir el Dockerfile de la aplicación real

Entra en la carpeta que contiene el proyecto MVC:

```
Push-Location .\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc
Get-Content .\Dockerfile
```

El Dockerfile de referencia contiene dos etapas y debe terminar con un comando que ejecute nuestra DLL:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS compilacion
WORKDIR /src
COPY . .
RUN dotnet restore CursoWeb.Mvc.csproj
RUN dotnet publish CursoWeb.Mvc.csproj -c Release --no-restore -o /app/publicada

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS ejecucion
```

```
WORKDIR /app
COPY --from=compilacion /app/publicada .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "CursoWeb.Mvc.dll"]
```

No se presenta como un bloque misterioso para copiar. Lee las instrucciones en orden:

- `FROM .../sdk:8.0` proporciona herramientas de compilación, como `dotnet restore` y `dotnet publish`.
- `WORKDIR /src` establece el directorio donde se manipularán los archivos.
- `COPY . .` introduce el contexto de construcción: **debemos ejecutar build desde la carpeta adecuada**.
- `RUN dotnet restore` obtiene las dependencias disponibles.
- `RUN dotnet publish` crea la salida para ejecución.
- La segunda etapa parte de `aspnet:8.0`, que incluye el runtime ASP.NET Core pero no necesita el SDK completo.
- `COPY --from=compilacion` introduce solo los artefactos publicados.
- `ASPNETCORE_URLS` permite que Kestrel escuche en 8080 dentro del contenedor.
- `EXPOSE 8080` documenta el puerto interno; no crea por sí solo un acceso desde Windows.
- `ENTRYPOINT` inicia la DLL cuando se ejecuta el contenedor.

El archivo `.dockerignore` excluye carpetas de compilación local (`bin`, `obj`) y archivos de desarrollo. Su función es reducir el contexto enviado a Docker y evitar introducir resultados anteriores que pudieran confundir la construcción.

Paso P5.5.3. Construir una imagen y distinguir build de run

Desde la misma carpeta, ejecuta:

```
docker build -t cursoweb-mvc:m05 .
```

El punto final indica que **el contexto de construcción es el directorio actual**. Si lo omites o ejecutas el comando desde una carpeta incompatible, Docker no encontrará necesariamente `CursoWeb.Mvc.csproj`.

Cuando termine sin errores, comprueba la imagen:

```
docker images cursoweb-mvc
```

Este resultado solo acredita construcción. Aún no hay navegador, proceso ASP.NET Core en ejecución ni respuesta HTTP. Si falla `dotnet restore` durante el build, investiga conectividad, fuentes NuGet y archivo de proyecto. No alteres el contenido de la API Blazor para resolver esa incidencia.

Paso P5.5.4. Ejecutar el contenedor con el modo SQL explícito

Docker Linux no contiene la instancia SQL Server Express LocalDB de Visual Studio. No podemos trasladarla copiando la cadena de conexión. El proyecto MVC M05 ofrece un modo **sin LocalDB**

destinado exclusivamente a ensayos de hospedaje; conserva el CRUD completo para el entorno Windows original.

Ejecuta:

```
docker run --name cursoweb-m05 --rm -p 8081:8080 -e Despliegue__SinLocalDb=true cursoweb-
mvc:m05
```

La opción `-e` define la configuración del proceso dentro del contenedor. `-p 8081:8080` publica el puerto del contenedor **8080** en el puerto del host **8081**. `--name` facilita identificar el proceso. `--rm` elimina el contenedor cuando termina; no elimina automáticamente la imagen.

Mantén esta terminal abierta. El proceso ASP.NET Core debe continuar escuchando. Si termina de inmediato, consulta la salida y los logs antes de modificar el Dockerfile.

Paso P5.5.5. Comprobar desde una segunda terminal y un navegador

Abre otra terminal PowerShell, sin detener el contenedor:

```
docker ps
curl.exe -i http://localhost:8081/
curl.exe -i http://localhost:8081/Productos/Estado
curl.exe -i http://localhost:8081/css/site.css
```

En un navegador abre `http://localhost:8081/` y comprueba que aparece la misma aplicación MVC. La ruta `/Productos/Estado` debe devolver el texto esperado. La hoja CSS debe servirse correctamente. Es importante anotar **la URL del host**, no el puerto 8080 que la aplicación utiliza dentro del contenedor.

Consulta también `http://localhost:8081/Productos`. El resultado previsto es **503**, con el mensaje explícito sobre LocalDB, porque el experimento no dispone de una base SQL Server. No declares probado el CRUD. Este punto demuestra publicación y ejecución del host MVC Linux, no persistencia ni migración de base de datos.

Paso P5.5.6. Consultar logs y reconocer el proceso real

En la segunda terminal:

```
docker logs cursoweb-m05
docker ps --filter "name=cursoweb-m05"
```

Los logs deben ayudar a localizar el inicio del servidor o posibles errores. Si `docker ps` no muestra el contenedor, comprueba si terminó y consulta su salida en la primera terminal. Un error de navegación no demuestra por sí solo que la imagen sea inválida; podríamos haber publicado un puerto distinto, dejado el proceso detenido o introducido una variable de configuración incorrecta.

Comprueba por separado tres afirmaciones: existe la imagen, está en ejecución un contenedor y su ruta HTTP devuelve una respuesta válida. Son evidencias complementarias.

Paso P5.5.7. Recuperar un puerto ocupado sin reescribir la aplicación

Si otro proceso ocupa el puerto 8081 **del host**, no es necesario cambiar la escucha interna 8080. Detén el contenedor propio y ejecútalo con un puerto del host libre, por ejemplo:

```
docker run --name cursoweb-m05 --rm -p 8082:8080 -e Despliegue__SinLocalDb=true cursoweb-
mvc:m05
```

La dirección que abrirás ahora será `http://localhost:8082/`. Esta comprobación permite explicar por qué el primer número de `-p` pertenece al host y el segundo al contenedor. No mantengas ejecutándose dos contenedores con el mismo nombre: Docker rechazaría esa duplicación.

Paso P5.5.8. Detener y retirar el contenedor de ensayo

Desde otra terminal, solicita:

```
docker stop cursoweb-m05
```

Debido a `--rm`, el contenedor debe desaparecer de la lista de activos tras detenerse. Comprueba:

```
docker ps --filter "name=cursoweb-m05"
docker images cursoweb-mvc
```

La imagen puede seguir existiendo. El docente decidirá si se conserva para repetir el ejercicio o se retira después de clase. No elimines imágenes de otros proyectos ni utilices comandos de limpieza masiva para completar este paso.

Regresa a la raíz del repositorio:

```
Pop-Location
```

Paso P5.5.9. Comparar las tres modalidades sin confundir garantías

Registra los resultados observados en esta matriz:

Pregunta	Kestrel Windows	IIS Windows	Docker Linux
¿Se ha publicado MVC A?	Publicación de P5.2	Carpeta IIS	Imagen multietapa
¿Quién inicia el host?	`dotnet`	IIS/ANCM	`ENTRYPOINT`
¿Qué puerto se abre desde el host?	5091	8091	8081
¿Qué ruta sin base has probado?	`/Productos/Estado`	`/Productos/Estado`	`/Productos/Estado`
¿Has comprobado el CRUD en LocalDB?	Solo si hiciste la prueba con base real	No en el ensayo simple	No
¿Se ha probado el mecanismo real?	Requiere ejecución Kestrel	Requiere IIS funcionando	Requiere `docker run` real

No debes completar una casilla con «sí» si la herramienta correspondiente no estaba instalada. Si solo leíste el Dockerfile, no construiste la imagen; si solo ejecutaste Kestrel, no probaste IIS; si se sirvió la portada sin base, no confirmaste CRUD persistente.

Paso P5.5.10. Reconstruir desde el principio el Dockerfile de la aplicación real

Relación con la teoría: apartados 5.5.8–5.5.10. El recorrido presencial ya ha probado la imagen preparada. **Esta reconstrucción forma parte del itinerario presencial esencial del P5.5:** el alumno debe escribir progresivamente su Dockerfile, no limitarse a leer el archivo de referencia. Las comparaciones adicionales posteriores sí son ampliaciones autónomas. No se inicia una tercera aplicación ni se pierde el estado entregado.

Desde la raíz del repositorio, detén el contenedor de la práctica anterior y prepara una copia de referencia:

```
Push-Location .\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc
Copy-Item .\Dockerfile .\Dockerfile.p55-referencia -Force
```

Abre el Dockerfile en Visual Studio, conserva el respaldo y sustituye temporalmente el contenido por el estado inicial de la **etapa de compilación**:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS compilacion
WORKDIR /src
COPY . .
```

Explica antes de continuar dónde estará `CursoWeb.Mvc.csproj` después de `COPY`. Este archivo todavía no representa una imagen que sirva HTTP. Hemos elegido herramientas de construcción y una carpeta de trabajo, pero no hemos publicado la aplicación ni definido el proceso que la ejecutará.

Añade dos instrucciones y estudia su propósito por separado:

```
RUN dotnet restore CursoWeb.Mvc.csproj
RUN dotnet publish CursoWeb.Mvc.csproj -c Release --no-restore -o /app/publicada
```

`restore` resuelve dependencias; `publish` prepara los archivos de ejecución. Ambas instrucciones se ejecutan durante `docker build`, no durante cada petición HTTP. Una restauración que falla no puede arreglarse modificando una vista Razor.

Ahora crea una **segunda etapa** para la ejecución:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS ejecucion
WORKDIR /app
COPY --from=compilacion /app/publicada .
```

Esta etapa usa el runtime ASP.NET Core y copia la publicación producida por el SDK. La imagen final no necesita transportar el SDK de compilación. Sin embargo, todavía falta decirle al contenedor qué proceso debe ejecutar.

Completa el archivo con las instrucciones de escucha, documentación del puerto y arranque:

```
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "CursoWeb.Mvc.dll"]
```

Comprueba el resultado final: debe existir una etapa SDK, otra etapa runtime, una copia de archivos publicados y un `ENTRYPOINT` que apunta a la DLL MVC. Si el nombre de la DLL fuera el de Blazor o el de la API de M04, habríamos roto la continuidad del curso.

Paso P5.5.11. Construir y ejecutar la variante creada por el alumno

Desde la misma carpeta del proyecto MVC:

```
docker build -t cursoweb-mvc:m05-reconstruida .
docker images cursoweb-mvc
```

El identificador `m05-reconstruida` diferencia esta imagen de la creada en la práctica principal. Si Docker informa de que no encuentra el proyecto, confirma que el contexto indicado con el punto corresponde a la carpeta que contiene el archivo `.csproj`.

Ejecuta ahora la publicación en un contenedor propio y un puerto de host diferente:

```
docker run --name cursoweb-m05-reconstruida --rm `
  -p 8083:8080 `
  -e Despliegue__SinLocalDb=true `
  cursoweb-mvc:m05-reconstruida
```

En una segunda terminal PowerShell consulta los tres tipos de recurso y la ruta limitada:

```
docker ps --filter "name=cursoweb-m05-reconstruida"
curl.exe -i http://localhost:8083/
curl.exe -i http://localhost:8083/Productos/Estado
curl.exe -i http://localhost:8083/css/site.css
curl.exe -i http://localhost:8083/Productos
```

La página inicial, Estado y CSS deben responder satisfactoriamente; Productos debe devolver 503 porque el modo del contenedor no dispone de LocalDB. El resultado permite comparar dos imágenes construidas a partir del mismo programa: la de referencia y la que acabas de reconstruir desde un Dockerfile progresivo.

Por qué no consideramos suficiente el resultado de `docker images`: una imagen disponible no garantiza que exista un proceso activo ni que el mapeo de puertos permita llegar a él. Hemos añadido una ejecución real y cuatro peticiones por ese motivo.

Detén el contenedor:

```
docker stop cursoweb-m05-reconstruida
```

Paso P5.5.12. Reconocer el contexto de construcción y sus exclusiones

Relación con la teoría: 5.5.9. Observa el `.dockerignore` situado junto al Dockerfile. Contiene exclusiones para los directorios locales `bin`, `obj` y archivos propios de Visual Studio. Su propósito es que el contexto de Docker transporte el material de origen necesario y no se confunda con salidas anteriores generadas en el equipo del alumno.

No es una alternativa a `dotnet restore` ni a `dotnet publish`. La etapa SDK sigue realizando estas operaciones dentro de la construcción. Una imagen que se construye correctamente después de

eliminar los artefactos locales de desarrollo muestra que no depende de una DLL compilada previamente en Windows.

Este laboratorio no necesita incorporar certificados privados ni credenciales al contexto. Se trabaja con el mismo proyecto MVC y datos de demostración. La imagen no incluye SQL Server LocalDB y la configuración de ejecución declara su ausencia para las rutas que lo necesitan.

Paso P5.5.13. Restaurar el Dockerfile canónico del curso

Una vez detenido el contenedor de la reconstrucción, vuelve a la terminal inicial, donde conservaste la copia del archivo:

```
Copy-Item .\Dockerfile.p55-referencia .\Dockerfile -Force
Remove-Item .\Dockerfile.p55-referencia
Pop-Location
```

Si habías dejado un proceso Docker ejecutándose, ciérralo antes de dar por terminada la sesión. La restauración asegura que la próxima persona encuentre el Dockerfile original y no un archivo incompleto del proceso de aprendizaje.

Explicación final del alumno: describe la diferencia entre las herramientas SDK de la primera etapa, el runtime de la segunda, el contexto entregado a Docker, el programa que inicia `ENTRYPOINT` y la URL pública del host. Debe poder señalar la función de cada elemento sin cambiar el código MVC.

Paso P5.5.14. Una avería distinta en cada etapa del recorrido

Si falla `docker build` al buscar el proyecto, revisa el contexto y la instrucción `COPY`: el Dockerfile no puede obtener automáticamente un archivo que no está en la estructura copiada.

Si la imagen existe pero no aparece un contenedor en `docker ps`, comprueba que ejecutaste `docker run` y que el proceso no terminó inmediatamente. La imagen no es un servidor activo por sí misma.

Si hay contenedor activo y el navegador no conecta, compara el puerto 8083 del host con el 8080 del contenedor y consulta los logs del proceso. No cambies HomeController para corregir una dirección equivocada.

Si `/` responde 200 y `/Productos` devuelve 503, la limitación coincide con `Despliegue__SinLocalDb=true`. No se ha sustituido SQL Server por SQLite ni se ha habilitado un CRUD ficticio.

El reto se considera comprendido cuando la respuesta a cada incidencia identifica qué capa está implicada y cuál es la corrección más pequeña. Reconocer las causas es más importante que memorizar el orden de los comandos sin entenderlo.

Paso P5.5.15. Interpretar el ciclo completo de un contenedor sin confundir etapas

Fundamento: 5.5.20 y 5.5.27. Hasta aquí el alumno ha construido y ejecutado manualmente la imagen. Ahora queremos reconocer el recorrido completo y las evidencias que aporta cada acción, sin crear una nueva solución ni cambiar el proveedor de datos.

Comienza en la carpeta de proyecto MVC A. Comprueba que el Dockerfile contiene las dos etapas originales y que la aplicación sigue registrando `UseSqlServer`. Revisa además la presencia de `.dockerignore`; durante el build Docker no necesita enviar los archivos `bin` y `obj` generados por Visual Studio.

Ejecuta `docker build -t cursoweb-mvc:m05 .` desde la carpeta que contiene el `.csproj`. Cuando termine, utiliza `docker images cursoweb-mvc` para confirmar la imagen. Todavía no abras el navegador: **una imagen no equivale a un contenedor ejecutándose**.

Inicia después el contenedor en un puerto host libre y con la variable que indica ausencia de LocalDB:

```
docker run --name cursoweb-m05 --rm `
  -e Despliegue__SinLocalDb=true `
  -p 8081:8080 `
  cursoweb-mvc:m05
```

Desde otra terminal comprueba `docker ps`. Si el contenedor no aparece, investiga su proceso; la ejecución puede haber terminado. Si aparece, solicita la ruta de estado del proyecto MVC A. Esta tercera observación demuestra que el puerto publicado permite llegar al proceso web.

Interpretación solicitada: ¿qué demostró el build?, ¿qué demostró `docker ps`? y ¿qué demuestra el 200 de `/Productos/Estado`? Las respuestas deben ser distintas. Si todo se resume como «Docker funciona», la actividad todavía no está comprendida.

Paso P5.5.16. Comparar los dos lados del mapeo sin modificar el Dockerfile

Fundamento: 5.5.24–5.5.25. El programa ASP.NET Core sigue escuchando en 8080 dentro del contenedor. Desde Windows hemos publicado ese puerto como 8081. El navegador consulta al host, no una dirección interna que deba adivinar.

Detén el contenedor anterior. Inicia otra instancia con el mismo proyecto e imagen, pero cambia solo el puerto exterior:

```
docker run --name cursoweb-m05 --rm `
  -e Despliegue__SinLocalDb=true `
  -p 8082:8080 `
  cursoweb-mvc:m05
```

Consulta `http://localhost:8082/Productos/Estado`. Debe seguir devolviendo el texto conocido. Ahora comprueba con `docker ps` qué relación de puertos se ha declarado. No edites `HomeController`, no modifiques el código de la API Blazor y no cambies la escucha 8080 del Dockerfile.

El resultado ilustra que una configuración de host puede cambiar sin alterar la aplicación. El primer número de `-p` pertenece a Windows; el segundo al contenedor. Si el alumno solicita 8081 después de publicar 8082, está utilizando la URL equivocada.

Paso P5.5.17. Entender una variante autónoma que comprueba varias condiciones

Fundamento: 5.5.27–5.5.29. El repositorio incluye un archivo PowerShell completo que automatiza una secuencia controlada del mismo laboratorio:

`M05_CONSOLIDACION_DESPLIEGUE/PROYECTO/VARIANTES/P5.5_DOCKER/ComprobarDockerMvc.ps1`

El contenido íntegro de la variante ejecutable es:

```
# P5.5 — Variante autónoma: ciclo completo sobre la MISMA aplicación MVC A.
# Desde la raíz del repositorio, en PowerShell y con Docker Linux disponible.
param(
    [string]$CarpetaMvc = ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc",
    [string]$Imagen = "cursoweb-mvc:m05-verificada",
    [string]$Contenedor = "cursoweb-m05-verificada",
    [int]$PuertoHost = 8083
)
$ErrorActionPreference = "Stop"
if (-not (Test-Path (Join-Path $CarpetaMvc "CursoWeb.Mvc.csproj"))) {
    throw "No existe el proyecto MVC en $CarpetaMvc"
}
$tipo = (docker info --format "{{.OSType}}").Trim()
if ($LASTEXITCODE -ne 0 -or $tipo -ne "linux") {
    throw "La práctica exige Docker operativo en modo Linux"
}
$yaExiste = docker ps -a --filter "name=^/$Contenedor$" --format "{{.Names}}"
if ($yaExiste) {
    throw "Ya existe un contenedor llamado $Contenedor. No se elimina automáticamente."
}
Push-Location $CarpetaMvc
try {
    docker build -t $Imagen .
    if ($LASTEXITCODE -ne 0) { throw "Falló docker build" }
} finally {
    Pop-Location
}
$creado = $false
try {
    docker run -d --name $Contenedor -p "${PuertoHost}:8080" `
        -e Despliegue__SinLocalDb=true $Imagen | Out-Null
    if ($LASTEXITCODE -ne 0) { throw "Falló docker run" }
    $creado = $true
    $base = "http://127.0.0.1:$PuertoHost"
    $listo = $false
    for ($i = 0; $i -lt 40; $i++) {
        Start-Sleep -Seconds 1
        try {
            $respuesta = Invoke-WebRequest "$base/Productos/Estado" -UseBasicParsing -
TimeoutSec 3
            if ($respuesta.StatusCode -eq 200) { $listo = $true; break }
        } catch {
            Write-Host "Esperando respuesta del contenedor..."
        }
    }
    if (-not $listo) {
        docker logs $Contenedor
        throw "El contenedor no devolvió 200 desde /Productos/Estado"
    }
    foreach ($ruta in @("/", "/Productos/Estado", "/css/site.css")) {
        $r = Invoke-WebRequest "$base$ruta" -UseBasicParsing -TimeoutSec 5
        if ($r.StatusCode -ne 200) { throw "Se esperaba 200 en $ruta" }
        Write-Host "${ruta}: 200"
    }
    try {
        Invoke-WebRequest "$base/Productos" -UseBasicParsing -TimeoutSec 5 | Out-Null
        throw "No es correcto obtener 200 del CRUD sin LocalDB"
    } catch [System.Net.WebException] {
        $estado = [int]$_.Exception.Response.StatusCode
        if ($estado -ne 503) { throw "Productos devolvió $estado en lugar de 503" }
```

```
        Write-Host "Productos: 503 previsto; LocalDB no existe en Linux"
    }
} finally {
    if ($creado) {
        docker logs $Contenedor
        docker stop $Contenedor | Out-Null
        docker rm -f $Contenedor | Out-Null
    }
}
Write-Host "Imagen conservada: $Imagen"
Write-Host "No se afirma funcionamiento del CRUD persistente."
```

El alumno puede modificar los parámetros antes de ejecutarlo, pero no debe suprimir las comprobaciones HTTP ni la limpieza controlada. El archivo de referencia conserva exactamente el código presentado y permite reconstruirlo sin fragmentos omitidos.

La variante no crea una segunda aplicación. Construye la imagen de MVC A, inicia un contenedor Linux, espera una respuesta de Estado, comprueba la portada y CSS, confirma un 503 esperado de Productos y detiene únicamente el contenedor que creó.

Se conserva como ampliación autónoma porque **primero** hay que entender cada comando manualmente. El script no sustituye el razonamiento. Su interés está en la forma de relacionar varias observaciones con mensajes específicos y en no dar por demostrada una función que no puede utilizar SQL Server LocalDB dentro de Linux.

Paso P5.5.18. Preparar el ensayo de la variante sin borrar contenedores ajenos

Antes de ejecutar el archivo adicional, confirma que Docker Desktop utiliza contenedores Linux y que el motor responde:

```
docker version
docker info --format "{{.OSType}}"
```

Debe estar disponible un servidor Docker y su tipo debe ser `linux`. Si el cliente existe pero no puede contactar con el motor, el ejercicio no puede continuar hasta que se haya preparado ese requisito del laboratorio.

Después comprueba si existe un contenedor llamado `cursoweb-m05-verificada`:

```
docker ps -a --filter "name=^/cursoweb-m05-verificada$"
```

Si existe uno anterior, no ejecutes la variante como si el nombre estuviera libre. Inspecciona de dónde procede ese recurso y solicita al docente instrucciones para conservarlo, renombrar la variante o retirarlo si pertenece exclusivamente al ensayo.

Este procedimiento evita que una automatización de aprendizaje borre recursos que no controla. El script comprueba el nombre y falla deliberadamente en vez de ejecutar una limpieza masiva.

Paso P5.5.19. Ejecutar la comprobación de imagen y HTTP desde la raíz

El archivo de variante acepta la carpeta de MVC, un nombre de imagen, un nombre de contenedor y un puerto del host. Con el motor Docker preparado y sin un nombre duplicado, ejecuta:

```
.\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\VARIANTES\P5.5_DOCKER\ComprobarDockerMvc.ps1 `
  -CarpetaMvc ".\M05_CONSOLIDACION_DESPLIEGUE\PROYECTO\MVC\CursoWeb.Mvc" `
  -Imagen "cursoweb-mvc:m05-verificada" `
  -Contenedor "cursoweb-m05-verificada" `
  -PuertoHost 8083
```

El script configura el contenedor con `Despliegue__SinLocalDb=true`. Debería confirmar HTTP 200 en la portada, en Estado y en CSS. Para `/Productos` exige 503. Si recibe 200 del CRUD, no lo presenta como éxito, porque el escenario carece de LocalDB y ese resultado requeriría investigar cómo se está atendiendo la ruta.

La variante incluye un bloque `finally` de limpieza. Su finalidad es detener y retirar **el contenedor con el nombre que acaba de crear**, no todas las imágenes del equipo. La imagen de este ensayo permanece disponible para repetirlo sin reconstruirla si así lo desea el alumno.

Paso P5.5.20. Interpretar un fallo sin falsear el resultado

Supongamos que el script no consigue conectar con la ruta de Estado tras iniciar el contenedor. No puede declarar éxito únicamente porque `docker run` devolvió un identificador. En ese caso consulta los logs y muestra un error. Las posibles causas incluyen un proceso que terminó, un problema de arranque de ASP.NET Core o una URL que no corresponde al puerto publicado.

Si la portada responde 200 pero el CSS devuelve 404, eso no demuestra que todo el alojamiento haya fallado. La acción MVC pudo atender correctamente mientras un recurso estático no estaba disponible. Revisaremos la publicación de `wwwroot` y su URL.

Si `/Productos` devuelve 503, el script reconoce ese resultado como coherente con el modo sin LocalDB. No modifica el archivo `.csproj`, no instala SQLite y no escribe un listado ficticio para conseguir una respuesta 200.

El alumno debe poder explicar cada condición del script con palabras propias. Un script que imprime «OK» sin haber enviado una petición HTTP no cumpliría la finalidad pedagógica del reto.

Paso P5.5.21. Distinguir la vida del contenedor de la permanencia de la imagen

Tras la ejecución, el contenedor de comprobación se detiene y se retira. Consulta:

```
docker ps -a --filter "name=^/cursoweb-m05-verificada$"
docker images cursoweb-mvc
```

La primera consulta no debería mostrar el contenedor de ensayo; la segunda puede seguir mostrando la imagen. Esta diferencia es normal: el contenedor era una instancia y la imagen es el artefacto reutilizable con el que fue creado.

El Dockerfile del proyecto también sigue existiendo, y el código MVC no se ha modificado. La eliminación de una instancia no equivale a eliminar la imagen ni a borrar el proyecto fuente. Por eso una práctica de limpieza no debe utilizar comandos indiscriminados que retiren recursos de otros cursos.

Paso P5.5.22. Completar una comparación final basada en resultados observados

Conserva un resumen de las tres modalidades que has probado, utilizando las mismas rutas del proyecto MVC A. Para Kestrel indica el puerto local elegido y la DLL publicada. Para IIS indica el sitio y binding y confirma si se ejecutó realmente la solicitud al servidor IIS. Para Docker indica el nombre de imagen, el contenedor y la relación de puertos.

En cada modalidad explica si `/Productos/Estado` respondió y por qué esa ruta es apropiada para verificar alojamiento sin depender de SQL. Indica también si el CRUD fue probado de verdad con LocalDB en Windows o si el ensayo estuvo limitado al modo sin base.

No se considera correcta una tabla que atribuya CRUD persistente a Docker Linux cuando solamente se ejecutaron la portada y CSS. La comparación debe conservar las limitaciones observadas.

Paso P5.5.23. Restaurar las condiciones de trabajo para otro alumno

Detén las terminales de aplicaciones que todavía queden abiertas. Confirma que el contenedor de esta práctica no permanece en ejecución. Si cambiaste alguna variable de entorno en PowerShell, elimínala en esa misma terminal. Si reconstruiste el Dockerfile durante P5.5.10, comprueba que el archivo de referencia se restauró.

La carpeta MVC A sigue siendo la acumulativa de M02. La API de productos y Blazor B permanecen como proyectos de la otra línea didáctica del curso. No borres sus carpetas al terminar un ensayo de Docker. Tampoco cambies el proveedor de base del CRUD original.

Resultado final de aprendizaje: el alumno puede explicar dónde se construyó la imagen, qué ejecutó el contenedor, qué dirección atiende ASP.NET Core y qué respuesta concreta demuestra funcionamiento. La publicación se entiende como un cambio de entorno para el mismo proyecto, no como la creación de una aplicación nueva.

Paso P5.5.24. Reconocer en Docker Desktop las mismas piezas vistas en consola

Fundamento: 5.5.20 y 5.5.27. El recorrido de consola ha enseñado las diferencias entre Dockerfile, imagen y contenedor. Ahora contrastaremos esas piezas con la interfaz gráfica de Docker Desktop, sin instalar una herramienta diferente ni crear otra aplicación.

Abre Docker Desktop y confirma que el motor está en ejecución con soporte para contenedores Linux. Las secciones de la interfaz pueden cambiar de nombre entre versiones; normalmente existe una vista de **Images** y otra de **Containers**. No memorices solo la ubicación visual: identifica qué tipo de recurso representa cada sección.

En la vista de imágenes localiza `cursoweb-mvc:m05`, creada durante P5.5.3. Debe aparecer aunque el contenedor de ejecución ya se haya detenido. El registro de una imagen no demuestra que haya un servidor escuchando; corresponde al artefacto que Docker utiliza como plantilla.

En la vista de contenedores busca `cursoweb-m05`, si la instancia continúa activa. Si terminaste la práctica con `--rm`, puede que ya no aparezca. Esa ausencia no significa que la imagen se haya eliminado. Separa con claridad el estado de la imagen del ciclo de vida del contenedor.

Paso P5.5.25. Iniciar un contenedor y relacionarlo con la vista gráfica

Desde PowerShell, situado en la carpeta de proyecto MVC si necesitas reconstruir la imagen, confirma su presencia:

```
docker images cursoweb-mvc
```

Si la imagen existe, puedes iniciar el mismo ensayo con un nombre identificable:

```
docker run --name cursoweb-m05-grafico --rm `
  -e Despliegue__SinLocalDb=true `
  -p 8084:8080 `
  cursoweb-mvc:m05
```

El puerto exterior 8084 se utiliza para distinguir esta instancia de otros ensayos. El proceso ASP.NET Core continúa escuchando internamente en 8080, tal como indica el Dockerfile.

Mientras la terminal permanece abierta, regresa a Docker Desktop. Identifica el contenedor `cursoweb-m05-grafico` y su estado de ejecución. Si el nombre no aparece, comprueba primero que `docker run` terminó correctamente y que la interfaz está conectada al mismo motor que la terminal.

No hagas clic sobre opciones de eliminación de recursos que no conozcas. El objetivo del ejercicio es observar una instancia propia, no limpiar el entorno Docker del equipo.

Paso P5.5.26. Comparar la dirección publicada en la interfaz y en la consola

Desde otra terminal solicita:

```
docker ps --filter "name=cursoweb-m05-grafico"
curl.exe -i http://localhost:8084/Productos/Estado
```

Debes observar el contenedor y recibir la respuesta de Estado de MVC A. Vuelve a Docker Desktop y busca la información de los puertos publicados para esa instancia. Relaciona el número exterior 8084 con el interior 8080.

Pregunta práctica: ¿qué ocurriría si abrieras `http://localhost:8080/` en Windows cuando la instancia publica 8084:8080? Estarías solicitando un puerto diferente del que declaraste para el host. El hecho de que exista un `EXPOSE 8080` no cambia el mapeo real.

No modifiques el Dockerfile para arreglar una URL que no corresponde al puerto exterior. Si necesitas usar otro puerto libre del host, detén el contenedor e inícialo de nuevo con otro primer número de `-p`.

Paso P5.5.27. Consultar los logs desde dos interfaces

Fundamento: 5.5.28. Con `cursoweb-m05-grafico` todavía activo, consulta la salida desde consola:

```
docker logs cursoweb-m05-grafico
```

La salida debería incluir información de ASP.NET Core y, si hubo solicitudes, posibles registros de ejecución. Después abre los detalles de ese mismo contenedor en Docker Desktop y localiza su

vista de logs. Compara si corresponde al proceso que está ejecutándose, evitando mezclarlo con otro contenedor anterior.

Los logs aportan información del proceso. No sustituyen la prueba HTTP: el contenedor puede estar activo y haber escrito mensajes de arranque aunque el usuario solicite un puerto equivocado. Por eso esta práctica mantiene tres observaciones independientes: contenedor activo, salida del proceso y respuesta al endpoint Estado.

Si el contenedor termina, consulta el error de la terminal y los logs que sigan accesibles. No atribuyas automáticamente la caída a una clase MVC; el problema puede haber ocurrido durante el arranque por una condición de entorno.

Paso P5.5.28. Inspeccionar sin afirmar que existe un CRUD Linux persistente

Con el contenedor accesible en 8084, solicita:

```
curl.exe -i http://localhost:8084/
curl.exe -i http://localhost:8084/css/site.css
curl.exe -i http://localhost:8084/Productos
```

Las dos primeras solicitudes deben servir la página inicial y un recurso estático. La ruta de productos debe devolver 503 por la configuración `Despliegue__SinLocalDb=true`.

Si la interfaz de Docker Desktop indica que el contenedor está sano desde el punto de vista de su ejecución, esa información no convierte el 503 en un catálogo SQL funcional. Debemos seguir distinguiendo el estado administrativo de la instancia y el comportamiento real de cada endpoint.

La base SQL Server Express LocalDB continúa perteneciendo al escenario Windows de MVC A. Ni Docker Desktop ni la publicación de la imagen Linux han instalado una instancia de esa tecnología dentro del contenedor.

Paso P5.5.29. Detener el contenedor sin eliminar la imagen

Regresa a la terminal que ejecuta el contenedor o utiliza una segunda para detener exactamente esta instancia:

```
docker stop cursoweb-m05-grafico
```

Como fue iniciada con `--rm`, debe retirarse después de detenerse. Comprueba en Docker Desktop que el estado se ha actualizado; no utilices sus controles de eliminación masiva.

En PowerShell confirma la diferencia:

```
docker ps -a --filter "name=cursoweb-m05-grafico"
docker images cursoweb-mvc
```

La imagen `cursoweb-mvc:m05` puede seguir existiendo. Esto demuestra que el objeto ejecutable que se retiró no era la imagen misma. Si el alumno desea iniciar de nuevo el proyecto, puede crear otra instancia a partir de la imagen sin volver a editar `HomeController`.

Paso P5.5.30. Resolver una discrepancia entre consola y Docker Desktop

Un alumno ejecuta `docker ps` y ve un contenedor, pero no logra localizarlo en la interfaz gráfica. Antes de declarar que Docker está roto, comprueba que ambos programas están conectados al mismo motor y contexto. La existencia de varios contextos Docker en algunos equipos puede hacer que las herramientas consulten entornos distintos.

En el laboratorio estándar, con Docker Desktop correctamente preparado, consola e interfaz deberían representar el mismo conjunto de recursos. Sin embargo, la ausencia de una tarjeta gráfica en una vista no constituye prueba suficiente de que el proceso web no exista: la petición HTTP y `docker ps` proporcionan evidencias adicionales.

La investigación debe identificar el nombre exacto del contenedor y el puerto declarado. No se debe reconstruir una imagen cada vez que no se encuentra un icono en la interfaz.

Paso P5.5.31. Comparar la gestión gráfica con la gestión de procesos Windows

Durante Kestrel ejecutamos una DLL en PowerShell y detuvimos su proceso con Ctrl+C. En IIS creamos un sitio y un Application Pool bajo la administración del servidor Windows. En Docker creamos un contenedor a partir de una imagen y lo gestionamos con Docker Desktop o comandos.

Las interfaces son diferentes, pero las preguntas siguen siendo comparables: qué programa se ejecuta, con qué recursos, en qué puerto y qué solicitudes puede atender. MVC A sigue utilizando sus mismos controladores y vistas.

El alumnado no necesita aprender a administrar un clúster de contenedores ni a configurar redes distribuidas para entenderlo. Debe relacionar el contenedor de Docker Desktop con un proceso ASP.NET Core y una publicación existente.

Paso P5.5.32. Cierre razonado de la ampliación gráfica

Antes de terminar, identifica en Docker Desktop una imagen del proyecto, explica por qué puede permanecer aunque no exista un contenedor activo, señala el nombre de la instancia de ensayo cuando está ejecutándose y comprueba la relación de puertos exterior/interior.

Desde el navegador verifica la portada MVC y desde PowerShell comprueba la ruta de Estado. Explica además la respuesta 503 del catálogo cuando la ejecución se realiza sin LocalDB. Es una limitación contractual del escenario, no una falla que deba ocultarse.

Detén y retira solamente la instancia creada por ti, comprueba que el proyecto fuente conserva su Dockerfile y que no has eliminado otras aplicaciones. Con esto habrás reconocido el mismo ciclo de vida tanto desde consola como desde una interfaz gráfica.

Paso P5.5.33. Caso integrador: localizar el servidor al que pertenece una respuesta

Fundamento: consolidación de 5.2–5.5. Al terminar la jornada, el alumno conoce varios puertos y varios procesos. Es posible que una pestaña antigua del navegador muestre una página válida de una instancia que no corresponde al último ensayo.

Este caso requiere identificar tres URLs distintas del mismo proyecto MVC A: la usada en Kestrel, la configurada en IIS y la publicada por Docker. No es necesario que las tres estén activas

simultáneamente. Puedes realizar las comprobaciones de forma secuencial y anotar los resultados en el momento correspondiente.

La secuencia de referencia utiliza:

```
Kestrel:  http://localhost:5091/
IIS:      http://localhost:8091/
Docker:   http://localhost:8081/
```

Los números son convenciones del laboratorio; si has tenido que utilizar puertos alternativos, adapta las solicitudes y anota cuál usaste realmente.

Para cada modalidad identifica **quién inicia el proceso**, **dónde están los artefactos** y **qué dirección atiende el navegador**. Si dos direcciones muestran la misma portada, explica por qué eso no demuestra que sean el mismo proceso. La coincidencia visual procede de que comparten una aplicación, no de que compartan necesariamente una instancia.

Paso P5.5.34. Caso integrador: la portada responde pero una dependencia falta

Arranca la publicación MVC mediante el mecanismo que quieras comparar, utilizando explícitamente el modo sin LocalDB. Solicita una portada y la ruta de productos.

Espera 200 en la portada y 503 en Productos. Después responde: ¿qué parte del despliegue ha quedado demostrada y qué parte no? Hemos demostrado que el host ASP.NET Core puede servir la portada en ese entorno; no hemos demostrado que el CRUD EF Core utilice la base real.

La misma interpretación corresponde al ensayo IIS y al contenedor Linux. La causa de la limitación está documentada en la configuración de aplicación y no requiere modificar `ProductoFormulario`, `AppDbContext` ni el modelo de producto.

Respuesta inadmisible: «Como el servidor devuelve alguna página, toda la aplicación está desplegada». **Respuesta también inadmisible:** «Como Productos devuelve 503, el servidor HTTP no responde». Ambas contradicen el conjunto de observaciones.

Paso P5.5.35. Caso integrador: un cambio de código no llega al navegador

Imagina que el docente modifica la vista `Home/Index.cshtml` del proyecto fuente mientras una versión anterior ya está publicada en IIS o empaquetada en una imagen Docker. El navegador continúa mostrando la versión previa.

Antes de reescribir la vista, identifica qué artefactos está utilizando el servidor. En IIS comprueba la ruta física y la fecha de la publicación. En Docker comprueba qué imagen originó el contenedor. En Kestrel comprueba desde qué carpeta se inició la DLL. Solo después decide si hay que publicar de nuevo, reconstruir la imagen o iniciar otro proceso.

El recorrido de P5.2 con dos carpetas de publicación ya mostró el comportamiento. Ahora la misma idea sirve para explicar un problema de despliegue sin añadir otra aplicación ni otro lenguaje.

Pregunta razonada: ¿por qué modificar una vista en Visual Studio no puede garantizar que un contenedor ya creado cambie de contenido? Porque el contenedor utiliza los archivos de la imagen construida anteriormente, no una conexión viva con la carpeta de desarrollo.

Paso P5.5.36. Caso integrador: clasificar tres errores de una misma URL

Considera tres resultados posibles al solicitar `/Productos/Estado`:

En el primero no existe un proceso accesible en el puerto consultado. El navegador no logra establecer conexión. Revisa la instancia, la escucha y el mapeo externo si se trata de Docker.

En el segundo el servidor responde 404. Ya existe comunicación HTTP, pero el recurso solicitado no se encontró. Comprueba que ejecutas la versión correcta de MVC y que has escrito la ruta prevista.

En el tercero el servidor responde 200 con `Catalogo MVC disponible`. Se ha demostrado una ruta de controlador específica. Aún falta comprobar otras funciones si queremos afirmarlas, especialmente las que dependen de SQL.

La diferencia enseña que un síntoma genérico como «no abre» no es información suficiente. Se necesitan dirección efectiva, código HTTP y contenido de la respuesta.

Paso P5.5.37. Caso integrador: comparar un CSS con una acción de controlador

Solicita tanto `/` como `/css/site.css` en cualquiera de las modalidades de hospedaje que tengas activas. Si la página principal responde y el CSS falta, revisa el directorio publicado y `wwwroot`. Si la ruta de Estado tampoco responde, puede existir un problema diferente en el host, el enrutamiento o la versión desplegada.

El experimento controlado de P5.4 con el CSS retirado mostró que un recurso estático puede faltar mientras MVC continúa funcionando. En Docker la causa puede ser que la imagen se construyó desde un contexto incorrecto o que no incluyó la publicación esperada. En Kestrel puede ser que se esté ejecutando una carpeta de salida incompleta.

El criterio de comprobación es común, aunque el mecanismo para localizar los archivos publicados cambie. No debemos atribuir todos los fallos visuales al controlador Home/Index.

Paso P5.5.38. Caso integrador: interpretar un puerto ocupado sin borrar recursos

Si Kestrel no inicia porque 5091 está ocupado, revisa qué proceso escucha. Si Docker no puede publicar el puerto exterior 8081, comprueba si existe otra instancia utilizando esa dirección. Si IIS no puede activar un binding en 8091, revisa los sitios que ya utilizan el puerto.

La corrección adecuada puede consistir en detener **un recurso propio** que ya no se necesite o en elegir un puerto alternativo. No debe consistir en borrar sitios ajenos, matar indiscriminadamente procesos o eliminar todos los contenedores del equipo.

Esta comparación vincula tres mecanismos a una misma noción de sistema operativo: las direcciones de escucha son recursos compartidos. La aplicación MVC no necesita nuevos controladores para resolver una colisión exterior.

Paso P5.5.39. Caso integrador: elegir la siguiente acción sin modificar código

Ante un fallo, elige primero una observación:

Evidencia inicial	Comprobación siguiente razonable
No hay respuesta HTTP	Proceso, puerto y dirección de escucha
404 de un endpoint MVC	URL, routing y publicación utilizada
HTML 200 con CSS 404	`wwwroot`, ruta de recurso y archivos publicados
503 en Productos con modo sin base	Confirmar limitación intencionada
Imagen Docker disponible sin contenedor	Revisar `docker run` y salida
Sitio IIS Started sin HTML	Revisar ANCM, runtime y carpeta física
Portada de una versión antigua	Identificar artefactos del proceso activo

No hay una única orden que repare todos estos escenarios. Cada comprobación busca información nueva acerca de una capa concreta. Esta es la competencia práctica central del módulo: relacionar evidencia con una decisión proporcionada.

Paso P5.5.40. Restituir la infraestructura compartida al terminar

Detén los procesos Kestrel que hayas arrancado y libera sus puertos. Si utilizaste Docker, comprueba que el contenedor del ensayo está detenido; no necesitas eliminar la imagen si el docente desea conservarla para futuras repeticiones. Si creaste un sitio IIS adicional, informa de su nombre, binding y pool para que el docente decida su conservación o retirada.

Restaura todos los archivos de proyecto que hubieras modificado temporalmente mediante variantes; no dejes un Dockerfile a medio construir ni una API que devuelve respuestas de diagnóstico. Conserva el proyecto MVC A con su proveedor SQL Server y la aplicación Blazor B con su servicio HTTP tipado.

La restauración garantiza que otra persona pueda empezar desde un estado conocido y reproducir los resultados del manual. El objetivo no es una limpieza destructiva de la máquina, sino retirar únicamente las condiciones temporales del ensayo.

Paso P5.5.41. Explicar el recorrido sin depender de las capturas de pantalla

Como síntesis final, describe una de las tres modalidades de hospedaje en un párrafo que relacione el proyecto, la publicación, el proceso o contenedor, el puerto y la respuesta HTTP. Añade después una segunda frase que identifique la dependencia de datos y su disponibilidad real.

Un ejemplo correcto para Docker diría que se construyó una imagen multietapa de MVC A, se inició un contenedor Linux que ejecuta ASP.NET Core y se comprobó la portada a través de un puerto publicado desde Windows. Añadiría que el CRUD no se probó porque el ensayo utilizó explícitamente el modo sin LocalDB.

El alumno que puede formular estas afirmaciones ha comprendido más que una receta de comandos: sabe delimitar el resultado de una publicación y evitar atribuirle capacidades no comprobadas. Con ello concluye el recorrido didáctico del M05 sin incorporar contenidos ajenos al temario acordado.

Ampliación autónoma P5.5: una avería diferente en cada capa

Estudia estos tres casos y expresa la acción correctiva más pequeña:

1. `docker build` falla porque no encuentra `.csproj`: ¿desde dónde has ejecutado el build y cuál es su contexto?
2. La imagen existe pero `http://localhost:8081` no responde: ¿hay contenedor activo?, ¿qué `-p` se usó?, ¿qué puerto escucha ASP.NET Core?
3. El inicio responde pero `/Productos` devuelve 503: ¿la ruta necesita LocalDB?, ¿qué configuración declaramos?, ¿es ese resultado coherente con el objetivo de la práctica?

La respuesta del tercer caso es deliberadamente distinta de «instalar SQLite» o «quitar EF Core». El proyecto base sigue siendo el que construimos en M02; el ensayo Docker solo demuestra que puede servirse una parte independiente de la infraestructura SQL Windows.

Cierre de M05: interpretar lo aprendido

No es necesario que las cinco tareas se conviertan en una aplicación empresarial ni que todos los servicios de M04 se ejecuten a la vez. Al finalizar, el estudiante debe distinguir:

- Una interfaz Blazor Interactive Server que consume una API mediante servicios tipados y estado de componente.
- Una API en memoria cuyo ciclo de vida no equivale a persistencia de base de datos.
- Una aplicación MVC con Razor y EF Core que se sigue utilizando sin reconstruirla.
- Compilación, ejecución y publicación como operaciones diferentes.
- Kestrel, IIS y Docker como opciones de hospedaje que mantienen el código MVC, pero cambian sus condiciones de arranque, puertos, permisos y dependencias.

Si un resultado no se observa, la práctica no se resuelve escribiendo «correcto» en una tabla. Se investiga el proceso, la ruta, la respuesta HTTP y las condiciones de la máquina y se declara con precisión qué parte ha podido comprobarse.